

SAMURAI Robot

МПС, объяснённая с нуля

Модель пространства состояний и МРС-регулятор
в коде, на пальцах и в формулах

Учебник-сопровождение к ветке feat/mps

Целевая аудитория:	студент 3–4 курса ИТ-вуза
Что нужно знать:	матан, линейная алгебра, основы Python
Что объясняется:	ТАУ, дифференциальный привод, ЛКР, МРС
Робот:	гусеничное шасси, 2 мотора, Raspberry Pi 4
Размерность модели:	$n = 5$ (состояние), $r = 2$ (управление)
Период дискретизации:	$T_s = 0.02$ с (50 Гц, секция mps:)
Стиль изложения:	story-driven — от сценария к формуле

Основано на:

Козлов В. Н., Куприянов В. Е., Шашихин В. Н.
«Теория автоматического управления» (СПбГПУ, 2008)

May 15, 2026

Contents

1	Зачем эта книжка	3
1.1	Кому это	3
1.2	Что нужно знать заранее	3
1.3	Чего книжка <i>не</i> объясняет	3
1.4	Как читать	4
1.5	Что значит «МПС»	4
1.6	Куда смотреть, если книжки мало	5
1.7	Обозначения	5
2	Робот глазами пользователя	6
2.1	Сценарий «D метров вперёд»	6
2.2	Что нужно роботу, чтобы это сделать	6
2.3	Три слоя проекта	7
2.4	Граничные правила	8
2.5	Что дальше	8
3	Дифференциальная кинематика	9
3.1	Вопрос	9
3.2	Геометрия	9
3.3	Команды робота: u_v и u_ω	10
3.4	Обратная задача: гусеницы $\rightarrow$ команды	10
3.5	Положение и курс в мировой системе	11
3.6	Где это в коде	11
3.7	Что мы получили	11
4	Откуда берётся состояние	13
4.1	Вопрос	13
4.2	Состояние в МПС-сценарии	13
4.3	Источники измерений	14
4.3.1	Одометрия по гусеницам	15
4.3.2	Курс по IMU	15
4.4	Слияние: position_fusion	15
4.5	Старт сценария: «обнуление» начала	16
4.6	Что мы получили	16
5	Динамика: моторы не отрабатывают мгновенно	17
5.1	Вопрос	17
5.2	Апериодическое звено первого порядка	17

5.3	Аналогично — угловая скорость	18
5.4	Полная нелинейная ОДУ-модель	18
5.5	Откуда берутся τ_v и τ_ω	19
5.6	Где это в коде	20
5.7	Что мы получили	20
6	Из нелинейной ОДУ в матрицы A_d, B_d	21
6.1	Вопрос	21
6.2	Выбор опорной точки	21
6.3	Ряд Тейлора	22
6.3.1	Вычисление матрицы A	22
6.3.2	Вычисление матрицы B	23
6.3.3	Численный пример	23
6.4	Управляемость	23
6.5	Дискретизация: ZOH	24
6.5.1	Аналитический вывод	24
6.5.2	Блочная экспонента (как считают в коде)	25
6.5.3	Полученная дискретная модель	25
6.6	Собственные значения	25
6.7	Каноническая МПС-форма $(s, v, \theta, \omega, e_{\text{int}})$	26
6.8	Что мы получили	27
7	Что такое регулятор	28
7.1	Вопрос	28
7.2	П-регулятор (proportional)	28
7.3	PID	29
7.4	Линейно-квадратичный регулятор (ЛКР)	29
7.4.1	Что значит «положительно определённая»	30
7.4.2	Зачем квадратичный штраф?	30
7.4.3	Результат ЛКР	30
7.5	Интуитивная карта	31
7.6	Связь с проектом	31
8	ЛКР: оптимальный регулятор	32
8.1	Вопрос	32
8.2	Постановка задачи	32
8.3	Принцип динамического программирования	32
8.3.1	Шаг 1: найти оптимальное $\mathbf{u}^*$	33
8.3.2	Шаг 2: уравнение для P (DARE)	33
8.4	Как решают DARE численно	34
8.5	Какие Q, R выбрать	34
8.5.1	Правило Брайсона	34
8.5.2	Дефолты проекта	35
8.5.3	Что меняется, если...	35
8.6	Замкнутая система	35
8.7	Слабое место: ограничения	36
8.8	Что мы получили	36
9	MPC: Model Predictive Control	37

9.1	Вопрос	37
9.2	Постановка	37
9.3	Лифтированная динамика	38
9.3.1	Стоимость в лифтированной форме	39
9.4	Квадратичное программирование (QP)	39
9.5	Два режима в проекте: clip и qr	40
9.5.1	clip (по умолчанию)	40
9.5.2	qr (полный)	41
9.6	Выбор горизонта N	41
9.7	Терминальный штраф $\mathbf{P}_f$	41
9.8	Слежение за reference	42
9.9	Sanity-check замкнутой системы	42
9.10	Hot-reload между прогонами	43
9.11	Что мы получили	43
10	Сценарий «D метров вперёд»	44
10.1	Что мы строим	44
10.2	Постановка задачи	44
10.3	Reference	45
10.4	Полный тик регулятора	45
10.5	Lifecycle прогона	46
10.6	Метрики качества	46
10.7	Safety	47
10.8	Симулятор	47
10.9	Что мы получили	48
11	Двухфазный сценарий: TURN, затем DRIVE	49
11.1	Вопрос	49
11.2	Фаза TURN	49
11.2.1	Reference	49
11.2.2	Принудительные ограничения фазы	50
11.2.3	Условие завершения TURN	51
11.2.4	Тайм-аут фазы TURN	51
11.3	Переход TURN $\rightarrow$ DRIVE	51
11.4	Фаза DRIVE	52
11.5	Timeline прогона	52
11.6	Граничные случаи	53
11.7	Зачем это всё было нужно	53
11.8	Что мы получили	54
12	3D-пикер цели: математика	55
12.1	Вопрос	55
12.2	Геометрия и соглашения	55
12.3	Точка клика $\rightarrow$ угол	56
12.3.1	Формула	56
12.3.2	Почему именно atan2?	56
12.3.3	Численные примеры	57
12.4	Угол $\rightarrow$ позиция маркера	57
12.4.1	Почему радиус <i>фиксированный</i> ?	57

12.5	Центральная мёртвая зона	58
12.6	Подпись угла	58
12.7	Полный пайплайн пикера	59
12.8	От UI к роботу	59
12.9	Тесты математики пикера	59
12.10	Что мы получили	60
13	Реализация: путь чисел	61
13.1	Вопрос	61
13.2	Этап 1: MATLAB (<code>matlab/main.m</code>)	61
13.3	Этап 2: YAML — источник правды	62
13.4	Этап 3: Python (<code>StateSpaceModel</code> , <code>MPCController</code>)	64
13.5	Live Apply: четвёртый канал	65
13.5.1	Поток данных	65
13.5.2	Что валидирует <code>Pi</code>	66
13.6	Согласование <code>sim</code> и работа	66
13.7	Тесты	66
13.8	Где какие риски	67
13.9	Что мы получили	67

Chapter 1

Зачем эта книжка

1.1 Кому это

Этот документ написан для студента, который услышал «модель пространства состояний» и «MPC» в курсе теории автоматического управления (ТАУ), открыл код проекта SAMURAI — и не понял, как одно стыкуется с другим. Учебник Козлова даёт строгие формулы, но не показывает, как эти формулы превращаются в команды моторам реального робота. Код проекта работает и моторы крутятся, но «почему именно эти числа в матрице **A**» не объясняет.

Книжка закрывает этот зазор. Она ведёт читателя по реальной траектории сценария «проехать D метров вперёд»: от нажатия кнопки в UI до ШИМ-сигнала в моторе. На каждом шаге появляется ровно та математика, которая нужна именно здесь, и тут же показывается, в каком файле проекта она живёт.

1.2 Что нужно знать заранее

- **Матан.** Производные, частные производные, ряд Тейлора первого порядка, неопределённый интеграл. Дифференциальные уравнения первого порядка с постоянными коэффициентами (типа $\dot{x} = -ax + bu$) — не пугают, но если пугают, то мы их разберём заново.
- **Линейная алгебра.** Что такое матрица, умножение матриц, транспонирование, собственные значения. Знать слово «положительно определённая» полезно, но не обязательно — введём.
- **Python на уровне чтения.** Уметь читать сигнатуру функции и срез массива `numpy`. Писать на нём не придётся — главы про реализацию показывают код проекта и комментируют его.

Если матан и ЛА давно не открывал — ничего страшного, нужные вещи вспомним по ходу.

1.3 Чего книжка *не* объясняет

Чтобы не разрастаться в энциклопедию, документ **намеренно не покрывает** следующее:

- Электронику и моторы: ШИМ, противо-ЭДС, H-мост. Считаем, что мотор — это «чёрный ящик», который получает команду и крутится с некоторой инерцией. Подробнее — в `latex_doc/chapters/14_pwm_motor.tex`.
- Фильтрацию датчиков: EKF на IMU, слияние акселерометра и одометрии. Положение робота уже считаем «измеренным», источник — глава 4. Подробнее — в `latex_doc/chapters/01_velocity_ekf.tex` и далее.
- SLAM, навигацию A*, Pure Pursuit, патрулирование. Сценарий «D метров вперёд» не требует карты и глобального плана — всё происходит в локальной системе координат робота.

1.4 Как читать

Главы пронумерованы по сценарию использования, а не по сложности математики. В каждой главе сначала возникает *вопрос* (например, «откуда робот знает, на сколько он повернулся?»), потом *интуиция* (на пальцах), потом *формула*, и в конце — *ссылка на файл и строку кода*, где это работает.

Если читаешь подряд, к главе 9 ты уже будешь знать, что такое **A**, **B**, **Q**, **R**, и MPC будет уже не «магия из учебника», а «вот зачем мы их вводили».

Если читаешь нелинейно (только нужное), вот ориентир по типам коробок:

- **Идея** — интуиция, можно читать без формул.
- **Формула** — ключевое уравнение, на которое потом ссылаемся.
- **Пример** — численный пример, можно «прожать» в голове.
- **В коде** — ссылка на файл/функцию в проекте.
- **Примечание** — объяснение «в сторону», не обязательное.
- **Что если?** — учебный «а что будет, если...». Полезно для интуиции.
- **Внимание** — типичная ошибка, реальный подводный камень проекта.

1.5 Что значит «МПС»

Идея

В этом проекте **МПС** = **Модель Пространства Состояний**, по-английски *state-space model*. Это способ описать поведение робота уравнением вида

$$\mathbf{x}_{k+1} = \mathbf{A}_d \mathbf{x}_k + \mathbf{B}_d \mathbf{u}_k,$$

где $\mathbf{x}$ — вектор состояния (что робот «есть в моменте»: где он, куда смотрит, как быстро едет), а $\mathbf{u}$ — вектор управления (команды скорости, которые мы ему даём).

Аббревиатура «МПС» **не** означает «методы принятия решений» (про Байеса, Гермейера, минимакс) — если ты искал это, тебе нужна другая ветка проекта.

1.6 Куда смотреть, если книжки мало

- `latex_doc/control_theory/` — более формальный, сухой вариант того же материала. Хорошо для зачёта.
- `latex_doc/main.tex` — общий справочник по всей математике робота (фильтры, SLAM, кватернионы, A^*) — 15 глав.
- `docs/mps/` — markdown-доки: `architecture.md`, `api.md`, `scenario_forward.md`. Описывают, как модуль устроен «снаружи».
- `docs/superpowers/specs/2026-05-05-mps-state-space-design.md` — источник правды по проектным решениям `feat/mps`.
- Учебник: Козлов В. Н., Куприянов В. Е., Шашихин В. Н. «Теория автоматического управления», СПбГПУ, 2008.

1.7 Обозначения

$\mathbf{x}(t) \in \mathbb{R}^n$	вектор состояния (непрерывное время)
$\mathbf{x}_k \in \mathbb{R}^n$	вектор состояния (дискретное время, шаг k)
$\mathbf{u}(t), \mathbf{u}_k$	вектор управления, r компонент
$\mathbf{y}(t), \mathbf{y}_k$	вектор выхода (что «видит» наблюдатель), m компонент
$\mathbf{A}, \mathbf{B}$	матрицы непрерывной модели $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$
$\mathbf{A}_d, \mathbf{B}_d$	матрицы дискретной модели $\mathbf{x}_{k+1} = \mathbf{A}_d\mathbf{x}_k + \mathbf{B}_d\mathbf{u}_k$
$\mathbf{C}, \mathbf{D}$	матрицы выхода: $\mathbf{y} = \mathbf{C}\mathbf{x} + \mathbf{D}\mathbf{u}$
$\mathbf{Q}, \mathbf{R}$	весовые матрицы критерия качества
$\mathbf{P}_f$	терминальный штраф (решение DARE)
$\mathbf{K}$	матрица обратной связи регулятора
T_s	период дискретизации (в проекте 0.02 с)
N	горизонт прогноза MPC
$\mathbf{x}^\top$	транспонирование
$\ \mathbf{x}\ _{\mathbf{Q}}^2$	сокращение для $\mathbf{x}^\top \mathbf{Q} \mathbf{x}$ (квадратичная форма)
$\bar{\mathbf{x}}, \bar{\mathbf{u}}$	опорная точка линеаризации
$\Delta \mathbf{x}$	отклонение от опорной точки: $\mathbf{x} - \bar{\mathbf{x}}$

Переменные с тильдой ($\tilde{x}$) встречаются в коде — это та же $\mathbf{x}$, просто иначе оформлено в идентификаторах.

Готов? Тогда поехали — смотрим, что вообще делает робот.

Chapter 2

Робот глазами пользователя

2.1 Сценарий «D метров вперёд»

Открываем веб-дашборд (`compute_node/frontend/`), идём на страницу `/mps`. Видим окружность с маркером, поля «*дистанция D*» и «*целевая скорость v_{target}* », кнопку «Run on Robot».

1. Двигаем маркер на окружности — выбираем **целевой курс φ** (на сколько градусов отвернуть от «прямо вперёд»).
2. Вводим D и v_{target} .
3. Жмём «Run on Robot».

Робот делает три вещи:

1. **Поворачивается на месте** на угол φ . Едет на нуле, крутится только разворот.
2. **Проезжает D метров** с целевой скоростью v_{target} , удерживая курс φ .
3. **Останавливается** и публикует результат: сколько проехал, какой был перерегулирующий выброс, как долго «успокаивался».

Всё. Один сценарий, конечный. Никакой карты, никакого глобального плана.

Идея

Книжка построена вокруг *этого* сценария. Каждый следующий вопрос — «а как робот...» — разбираем именно для этого сценария. Простота сценария — наше учебное преимущество: можно взглядеться в каждую формулу без отвлечения на другие задачи.

2.2 Что нужно роботу, чтобы это сделать

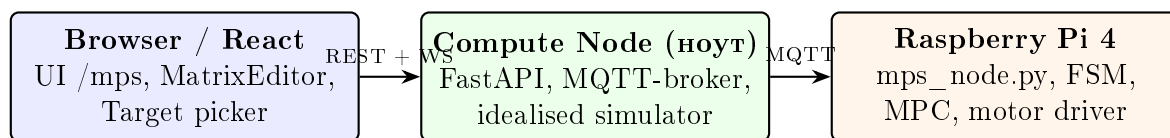
Распакуем то, что мы только что увидели. У робота должна быть способность ответить на пять вопросов.

(Q1) **Где я?** Чтобы понять, что проехал D метров, надо знать, на сколько я уже сдвинулся. Это *состояние позиции*. Об этом — глава 4.

- (Q2) **Куда я смотрю?** Чтобы повернуться к φ и не уехать боком, надо знать текущий курс θ . Это *состояние курса*. Тоже глава 4.
- (Q3) **Как мне сдвинуться/повернуться, если я скажу моторам «езжай»?** Если послать команду $u_v = 0.2$ м/с, робот не сразу станет ехать 0.2 м/с — моторы инерционны. Это *динамика*. Об этом — главы 3, 5.
- (Q4) **Какую команду подать, чтобы достичь цели?** Не любой набор команд приведёт к цели хорошо. Нужен *регулятор*. Об этом — главы 7, 8, 9.
- (Q5) **Когда остановиться и как сказать «готово»?** Это *логика сценария*, конечный автомат и подсчёт метрик. Об этом — главы 10, 11.

2.3 Три слоя проекта

Робот — не один компьютер, а три. Они общаются через сеть.



Что где живёт:

- **Browser** (React) — то, что видит человек: пикер курса, поля распределения, графики прогона. Жесть математики тут минимум, но есть — глава 12 разберёт.
- **Compute Node** (FastAPI на ноутбуке) — посредник: принимает команды от UI, шлёт их роботу по MQTT, сохраняет историю прогонов. Заодно умеет *симулировать* прогон без физического робота — это идеальный $\mathbf{x}_{k+1} = \mathbf{A}_d \mathbf{x}_k + \mathbf{B}_d \mathbf{u}_k$ без шумов. Полезно для UI и юнит-тестов.
- **Pi** (Python, MQTT) — собственно мозг робота. Подписан на топик mps/scenario/run; запускает тик-цикл 50 Гц, внутри которого *вся боевая математика* — считывание одометрии, шаг MPC, выдача команды моторам.

Где что лежит

compute_node/frontend/src/pages/MpsPage.tsx	UI страница
compute_node/frontend/src/lib/targetAngle.ts	математика пикера
compute_node/dashboard/routers/mps.py	REST + WS
compute_node/mps_runner.py	идеальный симулятор
pi_nodes/nodes/mps_node.py	оркестратор на Pi
pi_nodes/control/state_space_model.py	plant ($\mathbf{A}_d, \mathbf{B}_d$)
pi_nodes/control/mpc_controller.py	MPC ($\mathbf{Q}, \mathbf{R}, \mathbf{K}, \mathbf{Q_P}$)
config.yaml	секция mps: с матрицами
matlab/main.m	оффлайн-синтез матриц

2.4 Граничные правила

Перед тем как нырять в математику, зафиксируем правила игры, которые не выводятся, а просто заданы дизайном модуля. Если ты увидишь в коде проверку, которая кажется лишней — скорее всего, она охраняет одно из этих правил.

Примечание

1. MPC активен **только в FSM-состоянии** DRIVE_FORWARD_MPS. В любом другом (IDLE, SEARCHING, GRABBING, ...) команды моторам идут от *другого* контроллера (старый PID), а MPC ничего не делает.
2. Сценарий **финитный**: достижение цели, тайм-аут, abort или ошибка — любое из четырёх событий завершает прогон, и FSM возвращается в IDLE.
3. Между прогонами можно **пересобрать** матрицы (MQTT-команда mps/-matrices/set) — это и есть «учебный пульт». Во время прогона матрицы не меняются.
4. В состоянии DRIVE_FORWARD_MPS **cmd_vel** (команда моторам) приходит *только* от mps_node. Голосовые команды, детекции мяча и джойстик игнорируются.
5. Симулятор **идеальный**: $\mathbf{x}_{k+1} = \mathbf{A}_d \mathbf{x}_k + \mathbf{B}_d \mathbf{u}_k$ без шумов, без *motor lag*, без проскальзывания. Робот — неидеальный, разница между симом и реальным прогоном — это и есть «ошибка моделирования».

2.5 Что дальше

Дальше мы аккуратно ответим на пять вопросов (Q1)–(Q5) по очереди. Сначала → кинематика (глава 3): что значит «робот едет вперёд» формально. Потом — откуда состояние берётся (глава 4). Затем динамика, линеаризация, регуляторы, и наконец — сам сценарий.

Идём.

Chapter 3

Дифференциальная кинематика

3.1 Вопрос

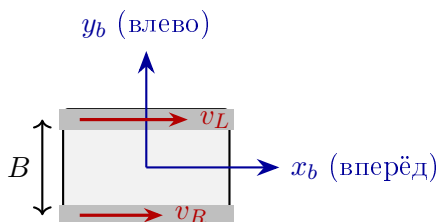
Робот — гусеничное шасси с **двумя задними моторами** (передние физически отсутствуют, см. `hardware_presets/samurai_default.yaml`). Левый мотор крутит левую гусеницу, правый — правую.

Идея

Мы хотим сказать роботу: «езжай со скоростью 0.2 м/с и поворачивай со скоростью 0.5 рад/с». Внутри у него — две гусеницы, которые крутятся каждая со своей скоростью. Откуда взять скорости гусениц из «езжай вперёд + поворачивай»?

3.2 Геометрия

Введём систему координат, связанную с роботом. Робот — это «жёсткий прямоугольник» шириной B метров (расстояние между серединами левой и правой гусениц, `wheel_base`). Ось x_b робота смотрит вперёд, ось y_b — влево.



Пусть гусеницы крутятся со скоростями v_L (левая) и v_R (правая) — это **линейные** скорости центров гусениц в продольном направлении, м/с.

3.3 Команды робота: u_v и u_ω

Команды роботу — не «крути левую и правую», а более удобная пара:

$$\mathbf{u} = \begin{pmatrix} u_v \\ u_\omega \end{pmatrix}, \quad u_v - \text{линейная скорость [м/с]}, \quad u_\omega - \text{угловая скорость [рад/с]}.$$

- $u_v > 0$ — ехать вперёд, $u_v < 0$ — назад.
- $u_\omega > 0$ — поворот *налево* (CCW против часовой, стандарт ROS и наш проект).
- $u_v = 0, u_\omega \neq 0$ — разворот на месте.
- $u_v \neq 0, u_\omega \neq 0$ — движение по дуге.

Прямая дифференциальная кинематика

Из геометрии: центр шасси едет со скоростью u_v , а каждая гусеница — на расстоянии $\frac{B}{2}$ от центра, поэтому добавочно скользит на $\pm \frac{B}{2} u_\omega$ (знак зависит от того, левая она или правая).

$$v_L = u_v + \frac{B}{2} u_\omega, \quad v_R = u_v - \frac{B}{2} u_\omega. \quad (3.1)$$

Примечание

Откуда знаки? При повороте налево ($u_\omega > 0$) робот «закидывает зад» вправо, его левая гусеница должна крутиться *быстрее* центра, а правая — медленнее. Поэтому к u_v добавляем $+\frac{B}{2}u_\omega$ для левой и вычитаем для правой.

3.4 Обратная задача: гусеницы $\rightarrow$ команды

Часто нужна обратная задача — по измеренным скоростям гусениц найти u_v и u_ω . Складываем и вычитаем (3.1):

$$u_v = \frac{v_L + v_R}{2}, \quad u_\omega = \frac{v_L - v_R}{B}. \quad (3.2)$$

Численный пример

$B = 0.17$ м. Хотим $u_v = 0.2$ м/с, $u_\omega = 0.5$ рад/с.

$$v_L = 0.2 + \frac{0.17}{2} \cdot 0.5 = 0.2 + 0.0425 = 0.2425 \text{ м/с},$$

$$v_R = 0.2 - 0.0425 = 0.1575 \text{ м/с}.$$

Левая гусеница на 30% быстрее правой $\rightarrow$ робот поворачивает налево.

3.5 Положение и курс в мировой системе

Команды u_v, u_ω — в *связанной* с роботом системе. Но позиция (p_x, p_y) и курс θ — в *мировой*. Чтобы получить их, нужно вспомнить, куда робот смотрит.

Идея

Если робот смотрит под углом θ к мировой оси x и движется вперёд со скоростью v , то проекции этой скорости на мировые оси — $v \cos \theta$ и $v \sin \theta$. Курс θ меняется со скоростью ω .

Кинематика робота в мировой СК

$$\begin{cases} \dot{p}_x = v \cos \theta, \\ \dot{p}_y = v \sin \theta, \\ \dot{\theta} = \omega. \end{cases} \quad (3.3)$$

Это **первое нелинейное звено** модели. Нелинейность здесь — из-за $\cos \theta$ и $\sin \theta$, зависящих от состояния.

3.6 Где это в коде

`pi_nodes/nodes/motor_node.py`

В микшере моторов (3.1) применяется к `cmd_vel`, полученному из MPC. Дальше скорости гусениц переводятся в ШИМ через калибровочные коэффициенты (РСА9685, каналы M1 ch10–11, M2 ch8–9). Подробнее — глава `latex_doc/chapters/14_pwm_motor.tex`.

`config.yaml`, секция `simulator`:

```
simulator:
  wheel_base: 0.17    # B [м]
  max_linear: 0.30    # |u_v|_max [м/с]
  max_angular: 2.00   # |u_omega|_max [рад/с]
```

3.7 Что мы получили

- Команда роботу — это пара (u_v, u_ω) , размерность $r = 2$.
- Мгновенно «исполнить» команду робот не может — сначала надо перевести её в скорости гусениц (тривиально, (3.1)), а потом — в ШИМ (см. отдельную главу). *Это всё ещё чистая кинематика без инерции.*
- Перевод в мировые координаты — через (3.3). Здесь появилась первая нелинейность модели.

В следующей главе разберёмся, *откуда* робот узнаёт свои p_x, p_y, θ (и можно ли им верить).

Chapter 4

Откуда берётся состояние

4.1 Вопрос

В (3.3) мы писали $\dot{p}_x = v \cos \theta$. Это дифференциальное уравнение — то, как состояние *меняется во времени*. Но МРС, который мы будем запускать 50 раз в секунду, ему на каждом тике нужно $\mathbf{x}_k$ — *измеренное состояние сейчас*.

Откуда оно? Робот же не «знает» своих координат — их нужно посчитать из сигналов реальных датчиков.

Идея

Это разделение — очень важное:

- *Модель* говорит, как состояние должно меняться (формула).
- *Оценка состояния* (state estimation) даёт текущее значение состояния по показаниям датчиков (число).

В этой главе нас интересует *оценка*. Модель будет дальше.

4.2 Состояние в МПС-сценарии

В сценарии «D метров вперёд» состояние — это пять чисел:

Канонический МПС-вектор состояния

$$\mathbf{x} = \begin{pmatrix} s \\ v \\ \theta \\ \omega \\ e_{\text{int}} \end{pmatrix} \in \mathbb{R}^5 \quad (4.1)$$

s	пройденная дистанция	м
v	продольная скорость	м/с
θ	курс (yaw)	рад
ω	угловая скорость	рад/с
e_{int}	интеграл ошибки скорости	м ($\approx v \cdot t$)

Примечание

Почему именно так, а не $(p_x, p_y, \theta, v, \omega)$?

В общей литературе по дифференциальным роботам состояние — $(p_x, p_y, \theta, v, \omega)$ (две декартовы координаты + курс + две скорости). И в проекте есть «легаси»-форма с такими пятью переменными (pi_nodes/control/state_space_model.py: переменные px, py, theta, v, omega). Но для сценария «D метров вперёд» эта форма *избыточна*: при правильной работе $p_y \equiv 0$ (робот едет прямо), а p_x совпадает с s .

Поэтому в МПС-сценарии мы переходим к более компактной форме (4.1): одна продольная координата s вместо двух, плюс *интегральный член* e_{int} , который ловит установившуюся ошибку по скорости (без него робот хронически не добывает v_{target}).

Внимание

В коде live ОБЕ модели существуют:

- Легаси $(p_x, p_y, \theta, v, \omega)$ — секция control. в config.yaml, используется старым PID-контуром.
- Каноническая $(s, v, \theta, \omega, e_{\text{int}})$ — секция mps:, используется в MPS-сценарии.

Это *не баг* — это разделение «учебной» подсистемы и боевой. Просто когда читаешь код, всегда смотри, *какая* секция (control.* или mps.*) грузит матрицы.

Историческая ловушка: mps_node в коммите 8521742 случайно строился из control.* и трактовал свой $x[1] = v$ как p_y , после чего МРС доворачивал робота, пытаясь свести p_y к v_{target} . Робот ездил по кругу.

4.3 Источники измерений

Каждая компонента (4.1) приходит не «магически», а из конкретного сенсора через конкретный фильтр.

Компонента	Сенсор	Преобразование
s, v	энкодеры моторов	интегрирование числа тиков $\rightarrow$ путь; velocity_ekf
θ, ω	MPU-6050 IMU (гироскоп)	ekf_imu: фильтр Калмана 1D на θ
e_{int}	—	накапливается <i>самим</i> mps_node как $\sum (v - v_{\text{target}}) \Delta t$

Подробное описание фильтров — в общем справочнике `latex_doc/chapters/01_velocity_ekf.tex`, `latex_doc/chapters/02_imu_ekf.tex`.

4.3.1 Одометрия по гусеницам

Энкодеры считают, сколько раз мотор провернулся за последний тик. Зная диаметр приводного колеса и передаточное число, это превращается в пройденный гусеницей путь $\Delta s_L, \Delta s_R$. Скорости и позиция центра:

$$v = \frac{\Delta s_L + \Delta s_R}{2 \Delta t}, \quad \omega_{\text{odom}} = \frac{\Delta s_L - \Delta s_R}{B \Delta t}, \quad s_{k+1} = s_k + v \Delta t. \quad (4.2)$$

Эти формулы — та же (3.2), что в главе 3, только применённая к *измеренным* Δs , а не к команде.

4.3.2 Курс по IMU

Гироскоп MPU-6050 даёт измеренную угловую скорость $\omega_{\text{imu}}(t)$. Интегрируя её, получим курс $\theta(t) = \int \omega_{\text{imu}} dt$. Проблема: гироскоп дрейфует. За минуту накопится несколько градусов ошибки даже на стоящем роботе.

Для коротких сценариев ($D \leq 5$ м $\Rightarrow t \leq 25$ с при $v_{\text{target}} = 0.2$ м/с) дрейф пренебрежимо мал, и мы используем прямое интегрирование с лёгкой ЕКФ-сглаживанием. Для длинных сессий проект делает дополнительную привязку через `position_fusion` (акселерометр + одометрия), но в МПС-сценарии это не критично.

4.4 Слияние: position_fusion

Все источники сводятся в одну ноду `position_fusion`, которая публикует MQTT-сообщение `odom` с полями `x`, `vx`, `theta`, `vz`. Это и есть «питающие» измерения для `mps_node`.

`pi_nodes/nodes/mps_node.py:342`

```

1 def _on_odom(self, topic: str, payload):
2     # samurai/{id}/odom — motor_node публикует:
3     # x в САНТИМЕТРАХ, vx в м/с, theta в рад, vz в рад/с.
4     s = float(payload.get('x', 0.0)) / 100.0 # см -> м
5     v = float(payload.get('vx', 0.0))
6     theta = float(payload.get('theta', 0.0))
7     omega = float(payload.get('vz', 0.0))
8     with self._lock:
9         self._x_meas = np.array(
10             [s, v, theta, omega, self._x_meas[_EINT]]
11         )
12         self._x_meas_ts = time.time()

```

Внимание

Подводный камень — единицы. `motor_node` публикует позицию в *сантиметрах*, а МПС-модель работает в *метрах*. В коммите 5f54c54 баг был такой: эвристика «если $|s| > 20$, то это уже метры, не конвертируй» оставляла первые 20 см не сконвертированными — и МРС видел фантомную ошибку позиции $\times 100$, прижимая u к насыщению $u_{\max}$. Исправлено безусловной конвертацией.

Урок: в любой межкомпонентной границе *явно фиксируй единицы*.

4.5 Старт сценария: «обнуление» начала

Когда пользователь жмёт «Run», `mps_node` запоминает *текущие* s и θ как $s_{\text{start}}, \theta_{\text{start}}$ (см. `_RunState`). Дальше состояние для МРС считается **относительно старта**:

$$x[s]_{\text{rel}} = s - s_{\text{start}}, \quad x[\theta]_{\text{rel}} = \text{wrap}_{[-\pi, \pi]}(\theta - \theta_{\text{start}}). \quad (4.3)$$

Идея

Зачем? Сценарий говорит: «проехать D метров *отсюда*». Если бы мы кормили в МРС *абсолютную* s , то reference s_{ref} тоже надо было бы делать абсолютным — а робот мог стартовать с $s = 4.7$ м после предыдущего прогона. Гораздо проще обнулить отсчёт в момент старта.

То же с курсом: $\theta_{\text{ref}} = \varphi$ означает «отвернуть на φ от *текущего* направления», а не от мирового севера.

Функция `_normalize_angle` оборачивает разность в $[-\pi, \pi]$, чтобы поворот на -179° не превратился в $+181^\circ$ (что бы сломало МРС, который квадратично штрафует ошибку курса).

4.6 Что мы получили

- Состояние МПС-сценария — пять чисел $(s, v, \theta, \omega, e_{\text{int}})$.
- Первые четыре приходят из MQTT-топика `odom` с Pi-ноды `position_fusion`, конвертируясь см→м.
- Пятый (e_{int}) живёт внутри самого `mps_node` — это интеграл ошибки скорости, нужен для устойчивого отслеживания v_{target} .
- На старте прогона s и θ *обнуляются*: дальнейшие вычисления идут в относительных координатах.

Теперь у нас есть и состояние, и формула (3.3), как оно меняется кинематически. Но кинематики мало: моторы инерционны, команда отрабатывается не мгновенно. Это — следующая глава.

Chapter 5

Динамика: моторы не отрабатывают мгновенно

5.1 Вопрос

В предыдущей главе мы кормили в кинематику v и ω так, будто что задали — то и едет. На самом деле это не так. Если в момент $t = 0$ сказать роботу «езжай 0.2 м/с» (раньше стоял), он не сразу достигнет 0.2 м/с — сначала будет разгоняться.

Идея

Между командой u_v и фактической скоростью v есть зазор: моторам нужно время, чтобы раскрутить инерцию, преодолеть противо-ЭДС и трение. Длительность этого зазора — постоянная времени τ_v . Это **динамика** робота. Без неё модель неполна, и МРС, обученный на одной только кинематике, будет систематически ошибаться.

5.2 Апериодическое звено первого порядка

Простейшая модель, которая адекватно описывает разгон моторов под такой нагрузкой — **апериодическое звено первого порядка** (в литературе также «инерционное звено», PT_1):

Динамика линейной скорости

$$\dot{v} = \frac{1}{\tau_v} (u_v - v). \quad (5.1)$$

Если u_v постоянно, решение этого уравнения — классическая экспонента:

$$v(t) = u_v - (u_v - v_0) e^{-t/\tau_v}, \quad (5.2)$$

где $v_0 = v(0)$. За время $t = \tau_v$ робот покрывает $\approx 63\%$ пути от v_0 до u_v ; за $t = 3\tau_v$ — $\approx 95\%$.

Идея

Можно прочесть (5.1) «по-человечески»: *скорость изменения скорости пропорциональна тому, как сильно она отстаёт от команды*. Чем больше отставание — тем сильнее «толкаем»; когда v догнала u_v — толкать перестаём. Аналогия: вода, текущая в бак, где скорость наполнения пропорциональна разности уровней. RC-цепь в электронике — та же самая динамика, потому её и называют «RC-звено».

Численный пример

Пусть $\tau_v = 0.15$ с (типичное значение для шасси SAMURAI). Робот стоит ($v_0 = 0$), команда $u_v = 0.2$ м/с.

t , с	v , м/с	доля от u_v
0.05	$0.2 \cdot (1 - e^{-1/3}) \approx 0.057$	28.3%
0.15	$0.2 \cdot (1 - e^{-1}) \approx 0.126$	63.2%
0.30	$0.2 \cdot (1 - e^{-2}) \approx 0.173$	86.5%
0.45	$0.2 \cdot (1 - e^{-3}) \approx 0.190$	95.0%

За $T_s = 0.05$ с (один тик MPC) робот успевает разогнаться только до 28% желаемого. Это и есть то, что MPC обязан знать.

5.3 Аналогично — угловая скорость

Динамика угловой скорости

$$\dot{\omega} = \frac{1}{\tau_{\omega}} (u_{\omega} - \omega). \quad (5.3)$$

Постоянная $\tau_{\omega} \approx 0.10$ с заметно *меньше* τ_v : крутить шасси на месте проще, чем разгонять центр массы.

5.4 Полная нелинейная ОДУ-модель

Соберём кинематику (3.3) и динамику (5.1), (5.3) в один вектор. Возьмём «физическое» (легаси) состояние из главы 4:

$$\mathbf{x}(t) = (p_x, p_y, \theta, v, \omega)^T \in \mathbb{R}^5, \quad \mathbf{u}(t) = (u_v, u_{\omega})^T \in \mathbb{R}^2.$$

Нелинейная ОДУ-модель робота

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}, \mathbf{u}) = \begin{pmatrix} v \cos \theta \\ v \sin \theta \\ \omega \\ \frac{1}{\tau_v} (u_v - v) \\ \frac{1}{\tau_\omega} (u_\omega - \omega) \end{pmatrix}. \quad (5.4)$$

Здесь:

- Первые три строки — *кинематика*. Нелинейность через $\cos, \sin$.
- Последние две — *динамика моторов*. *Линейная*: правая часть линейна по v, ω, u_v, u_ω .

Вся нелинейность сосредоточена в куске «как v и θ порождают движение в мире». Это важно: при линеаризации (следующая глава) именно этот кусок упростится.

5.5 Откуда берутся τ_v и τ_ω

Постоянные времени — *физические* параметры, оценённые экспериментально. В `test-s/test_motor_lag.py` есть тест, который гонит робот на серии импульсов u_v и подбирает τ_v по методу наименьших квадратов из закона (5.2). В `config.yaml` они зашиты как:

`config.yaml`, секция `control.plant`

```
control:
  plant:
    Ts:    0.05    # период дискретизации, с
    v0:    0.20    # опорная скорость для линеаризации, м/с
    tau_v: 0.15    # постоянная времени линейной скорости, с
    tau_w: 0.10    # постоянная времени угловой скорости, с
```

Что если?

Что если у моторов больше инерция? Тогда τ_v увеличится, и в дискретной модели $1/\tau_v$ уменьшится — собственное значение $\lambda = -1/\tau_v$ ближе к нулю. В дискретном виде $|\tilde{\lambda}| = e^{-T_s/\tau_v}$ ближе к единице — мотор «медленнее забывает» прошлую команду. МРС должен это учесть, выбирая более «нежную» **R**.

Если поставить в `config` *неправильно большое* τ_v , ничего не сломается — модель просто будет «думать, что моторы вялые». МРС будет давать более резкие команды (чтоб компенсировать выдуманную инерцию) — робот будет дёргаться. Хороший пример «учебной катастрофы» из задачи курсовой.

5.6 Где это в коде

pi_nodes/control/state_space_model.py:29

```

1 def _continuous_AB(v0: float, tau_v: float, tau_w: float):
2     """Build linearised A, B around forward motion at v0."""
3     A = np.array([
4         [0.0, 0.0, 0.0, 1.0, 0.0],
5         [0.0, 0.0, v0, 0.0, 0.0],
6         [0.0, 0.0, 0.0, 0.0, 1.0],
7         [0.0, 0.0, 0.0, -1.0/tau_v, 0.0],
8         [0.0, 0.0, 0.0, 0.0, -1.0/tau_w]
9     ])
10    B = np.array([
11        [0.0, 0.0],
12        [0.0, 0.0],
13        [0.0, 0.0],
14        [1.0/tau_v, 0.0],
15        [0.0, 1.0/tau_w]
16    ])
17    return A, B

```

Это уже *линеаризованная* версия модели — но мы её ещё не вывели. Чтобы её получить, нужно сделать линеаризацию (5.4) вокруг опорной точки. Этим займёмся в следующей главе.

5.7 Что мы получили

- Моторы — *инерционные*: команда отрабатывается за $\tau_v \sim 0.15$ с, $\tau_w \sim 0.10$ с.
- Динамика моторов — линейная: $\dot{v} = (u_v - v)/\tau_v$ и аналогично для ω .
- Полная модель робота (5.4) — пятимерная, нелинейная (только в кинематической части), с двумя управлениями.
- Постоянные времени — из тестов на реальном железе, сидят в config.yaml.

Дальше эту нелинейную систему надо превратить в линейную (для МРС и ЛКР это базовая необходимость). Идём.

Chapter 6

Из нелинейной ОДУ в матрицы $\mathbf{A}_d, \mathbf{B}_d$

6.1 Вопрос

Нелинейная модель (5.4) — честная, но **нерешаемая аналитически** и плохо подходящая для оптимизаторов. ЛКР и МРС, которые мы будем строить дальше, работают *только с линейными моделями* в дискретном времени:

$$\mathbf{x}_{k+1} = \mathbf{A}_d \mathbf{x}_k + \mathbf{B}_d \mathbf{u}_k.$$

Надо превратить (5.4) в эту форму.

Идея

Две операции, обе из учебника Козлова:

1. **Линеаризация.** Разложение в ряд Тейлора первого порядка вокруг *опорной точки* (равномерное движение вперёд). Получаем *непрерывные* матрицы $\mathbf{A}, \mathbf{B}$.
2. **Дискретизация.** Переход от $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$ к $\mathbf{x}_{k+1} = \mathbf{A}_d\mathbf{x}_k + \mathbf{B}_d\mathbf{u}_k$ через ЗОН (zero-order hold).

В этой главе пройдем по обеим.

6.2 Выбор опорной точки

Опорная точка (operating point, англ. *trim point*) — это тот режим, вокруг которого мы линеаризуем. Выберем его так, чтобы он был *реальной целью* сценария: робот едет вперёд с постоянной скоростью v_0 , ничего не вращая.

$$\bar{\mathbf{x}} = (\bar{p}_x, \bar{p}_y, \underbrace{0}_{\bar{\theta}}, v_0, \underbrace{0}_{\bar{\omega}})^T, \quad \bar{\mathbf{u}} = (v_0, 0)^T. \quad (6.1)$$

Подставим $\bar{\mathbf{x}}, \bar{\mathbf{u}}$ в (5.4):

$$\mathbf{f}(\bar{\mathbf{x}}, \bar{\mathbf{u}}) = (v_0 \cos 0, v_0 \sin 0, 0, (v_0 - v_0)/\tau_v, (0 - 0)/\tau_\omega)^\top = (v_0, 0, 0, 0, 0)^\top.$$

Координата p_x растёт со скоростью v_0 — логично; всё остальное не меняется. Опорная точка корректна: при $\mathbf{x} = \bar{\mathbf{x}}$, $\mathbf{u} = \bar{\mathbf{u}}$ робот действительно равномерно едет вперёд.

6.3 Ряд Тейлора

Пусть $\Delta \mathbf{x} = \mathbf{x} - \bar{\mathbf{x}}$, $\Delta \mathbf{u} = \mathbf{u} - \bar{\mathbf{u}}$ — *малые* отклонения от опорной. Разложим $\mathbf{f}$ в ряд:

$$\mathbf{f}(\mathbf{x}, \mathbf{u}) = \underbrace{\mathbf{f}(\bar{\mathbf{x}}, \bar{\mathbf{u}})}_{\dot{\bar{\mathbf{x}}}} + \underbrace{\left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_*}_{\mathbf{A}} \Delta \mathbf{x} + \underbrace{\left. \frac{\partial \mathbf{f}}{\partial \mathbf{u}} \right|_*}_{\mathbf{B}} \Delta \mathbf{u} + O(\|\Delta\|^2).$$

Вычитая $\dot{\bar{\mathbf{x}}}$ из обеих частей и обозначив $\Delta \dot{\mathbf{x}} = \dot{\mathbf{x}} - \dot{\bar{\mathbf{x}}}$, отбрасывая квадратичные члены, получаем:

Линеаризованная модель

$$\Delta \dot{\mathbf{x}}(t) = \mathbf{A} \Delta \mathbf{x}(t) + \mathbf{B} \Delta \mathbf{u}(t). \quad (6.2)$$

Важно: формула (6.2) верна *только в окрестности $\bar{\mathbf{x}}, \bar{\mathbf{u}}$* . Если робот сильно отклонится от опорной траектории (например, развернулся на 90°), линейная модель станет заметно врать.

6.3.1 Вычисление матрицы $\mathbf{A}$

Матрица Якоби $\mathbf{A} = \partial \mathbf{f} / \partial \mathbf{x}$ — это матрица 5×5 со столбцами $\partial \mathbf{f} / \partial x_j$. Считаем частные производные функций $f_1, \dots, f_5$ по $x_j \in \{p_x, p_y, \theta, v, \omega\}$ и подставляем опорные значения $\bar{\theta} = 0$, $\bar{v} = v_0$.

Строка 1: $f_1 = v \cos \theta$.

$$\left. \frac{\partial f_1}{\partial p_x} \right|_* = 0, \quad \left. \frac{\partial f_1}{\partial p_y} \right|_* = 0, \quad \left. \frac{\partial f_1}{\partial \theta} \right|_* = -v \sin \theta|_0 = 0, \quad \left. \frac{\partial f_1}{\partial v} \right|_* = \cos \theta|_0 = 1, \quad \left. \frac{\partial f_1}{\partial \omega} \right|_* = 0.$$

Строка 2: $f_2 = v \sin \theta$.

$$\left. \frac{\partial f_2}{\partial \theta} \right|_* = v \cos \theta|_0 = v_0, \quad \left. \frac{\partial f_2}{\partial v} \right|_* = \sin \theta|_0 = 0.$$

Остальные — нули.

Строка 3: $f_3 = \omega$. $\partial f_3 / \partial \omega = 1$. Остальные — нули.

Строка 4: $f_4 = (u_v - v) / \tau_v$. $\partial f_4 / \partial v = -1 / \tau_v$. Остальные — нули.

Строка 5: $f_5 = (u_\omega - \omega) / \tau_\omega$. $\partial f_5 / \partial \omega = -1 / \tau_\omega$. Остальные — нули.

Собираем:

Матрица $\mathbf{A}$ (непрерывная)

$$\mathbf{A} = \begin{pmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & v_0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & -1/\tau_v & 0 \\ 0 & 0 & 0 & 0 & -1/\tau_\omega \end{pmatrix}. \quad (6.3)$$

6.3.2 Вычисление матрицы $\mathbf{B}$

Аналогично для $\mathbf{B} = \partial \mathbf{f} / \partial \mathbf{u}$, размер 5×2 :

$$\frac{\partial f_4}{\partial u_v} = \frac{1}{\tau_v}, \quad \frac{\partial f_5}{\partial u_\omega} = \frac{1}{\tau_\omega}.$$

Остальные элементы — нули.

Матрица $\mathbf{B}$ (непрерывная)

$$\mathbf{B} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 1/\tau_v & 0 \\ 0 & 1/\tau_\omega \end{pmatrix}. \quad (6.4)$$

Если сравнить эти матрицы с кодом `state_space_model.py:29` (глава 5, листинг) — буквально одно и то же. *Это и есть наша модель.*

6.3.3 Численный пример

При $v_0 = 0.2$ м/с, $\tau_v = 0.15$ с, $\tau_\omega = 0.10$ с:

$$\mathbf{A} = \begin{pmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0.2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & -6.667 & 0 \\ 0 & 0 & 0 & 0 & -10 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 6.667 & 0 \\ 0 & 10 \end{pmatrix}. \quad (6.5)$$

6.4 Управляемость

Перед тем как строить регулятор, надо убедиться, что робот *в принципе* может быть приведён из любого состояния в любое другое. Это свойство **управляемости**.

Критерий Калмана

Объект полностью управляем тогда и только тогда, когда матрица управляемости имеет полный ранг:

$$\mathbf{S}_y = [\mathbf{B} \mid \mathbf{A}\mathbf{B} \mid \mathbf{A}^2\mathbf{B} \mid \dots \mid \mathbf{A}^{n-1}\mathbf{B}], \quad \text{rank } \mathbf{S}_y = n. \quad (6.6)$$

Для нашей $n = 5$, $r = 2$: $\mathbf{S}_y$ имеет размер 5×10 . Численная проверка в MATLAB (`matlab/analyze_system.m`) даёт $\text{rank } \mathbf{S}_y = 5$ при $v_0 \neq 0$ — система полностью управляема.

Внимание

При $v_0 = 0$ опорная точка вырождена. В (6.3) второй столбец, отвечающий за связь $\dot{p}_y$ с θ , обнуляется — координата p_y становится неуправляемой через малые отклонения $\Delta\theta$. Физически: стоя на месте, робот не умеет «поехать вбок», не повернувшись сильно (а это уже не «малые отклонения», и линейная модель не работает).

Следствие: линеаризуем только вокруг $v_0 > 0$. Чтобы стартовать с нуля, мы используем *два разных режима*: TURN на стоящем роботе управляется тем же MPC, но с *другой опорной* (об этом — глава 11).

6.5 Дискретизация: ZOH

MPC работает с дискретной моделью, потому что:

- Цикл управления — дискретный (тик T_s).
- Команды моторам тоже подаются дискретно: «на интервале $[kT_s, (k+1)T_s)$ команда $\mathbf{u}_k$, потом следующая».

Идея

ZOH (zero-order hold). «Нулевого порядка удержание»: команда $\mathbf{u}_k$ *постоянна* на всём интервале $[kT_s, (k+1)T_s)$. Это точно соответствует тому, как мы кормим `cmd_vel` моторам — между тиками значение не меняется. В этом предположении уравнение $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$ можно проинтегрировать *точно* от kT_s до $(k+1)T_s$ — получим формулу с матричной экспонентой.

6.5.1 Аналитический вывод

Решение линейной ОДУ с постоянной $\mathbf{u}_k$ на $[t_0, t_0 + T_s]$:

$$\mathbf{x}(t_0 + T_s) = e^{\mathbf{A}T_s} \mathbf{x}(t_0) + \int_0^{T_s} e^{\mathbf{A}(T_s - \tau)} \mathbf{B} d\tau \cdot \mathbf{u}_k.$$

Сменой переменной $\sigma = T_s - \tau$ интеграл превращается в

$$\int_0^{T_s} e^{\mathbf{A}\sigma} d\sigma \cdot \mathbf{B}.$$

Поэтому:

ЗОН-дискретизация

$$\mathbf{A}_d = e^{\mathbf{A}T_s}, \quad \mathbf{B}_d = \left(\int_0^{T_s} e^{\mathbf{A}\sigma} d\sigma \right) \mathbf{B}. \quad (6.7)$$

6.5.2 Блочная экспонента (как считают в коде)

Численно интеграл считать неудобно — есть **трюк** с блочной матричной экспонентой. Соберём блочную матрицу

$$\mathbf{M} = \begin{pmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{0} & \mathbf{0} \end{pmatrix} \in \mathbb{R}^{(n+r) \times (n+r)}.$$

Аккуратное матричное возведение в экспоненту даёт:

$$\exp(\mathbf{M}T_s) = \begin{pmatrix} e^{\mathbf{A}T_s} & \int_0^{T_s} e^{\mathbf{A}\sigma} d\sigma \cdot \mathbf{B} \\ \mathbf{0} & \mathbf{I}_r \end{pmatrix} = \begin{pmatrix} \mathbf{A}_d & \mathbf{B}_d \\ \mathbf{0} & \mathbf{I}_r \end{pmatrix}. \quad (6.8)$$

То есть: **одна** операция `expm` даёт сразу обе матрицы.

pi_nodes/control/state_space_model.py:48

```

1 def zoh_discretize(A, B, Ts):
2     """Exact ZOH discretization via block matrix exponential."""
3     n, r = A.shape[0], B.shape[1]
4     M = np.zeros((n + r, n + r))
5     M[:n, :n] = A
6     M[:n, n:] = B
7     M_d = expm(M * Ts)
8     A_d = M_d[:n, :n]
9     B_d = M_d[:n, n:]
10    return A_d, B_d

```

6.5.3 Полученная дискретная модель**Дискретная модель робота**

$$\mathbf{x}_{k+1} = \mathbf{A}_d \mathbf{x}_k + \mathbf{B}_d \mathbf{u}_k. \quad (6.9)$$

Это финальная форма, с которой будут работать ЛКР, МРС, симулятор.

6.6 Собственные значения

Собственные значения говорят, как себя ведёт открытая (без управления) система.

Примечание

Связь между непрерывным λ и дискретным $\tilde{\lambda}$:

$$\tilde{\lambda}_i = e^{\lambda_i T_s}.$$

Устойчивость:

- непрерывная: $\text{Re } \lambda_i < 0$ (левая полуплоскость);
- дискретная: $|\tilde{\lambda}_i| < 1$ (внутри единичного круга).

Для нашей $\mathbf{A}$ (6.3) собственные значения легко считать «руками» — $\mathbf{A}$ блочно-треугольная:

$$\det(\mathbf{A} - \lambda \mathbf{I}) = (-\lambda)^3 \cdot (-1/\tau_v - \lambda) \cdot (-1/\tau_\omega - \lambda).$$

$$\lambda_{1,2,3} = 0, \quad \lambda_4 = -1/\tau_v, \quad \lambda_5 = -1/\tau_\omega. \quad (6.10)$$

Три нуля — это *интеграторы*: p_x, p_y, θ накапливаются во времени без внутреннего «возврата к нулю». Два отрицательных корня — инерционность моторов: возмущение по скорости затухает.

В дискретной форме: $\tilde{\lambda}_{1,2,3} = 1$ (на границе устойчивости), $\tilde{\lambda}_4 = e^{-T_s/\tau_v} \approx 0.717$, $\tilde{\lambda}_5 = e^{-T_s/\tau_\omega} \approx 0.607$.

Внимание

Объект не асимптотически устойчив. Три собственных значения на единичной окружности означают: без управления отклонения по координатам и курсу не затухают (что логично — робот не «возвращается домой» сам по себе). Это норма для движущихся объектов. ЛКР/МРС сделают замкнутую систему устойчивой.

6.7 Каноническая МПС-форма $(s, v, \theta, \omega, e_{\text{int}})$

Выше мы работали с легаси-формой $(p_x, p_y, \theta, v, \omega)$, потому что её удобно линеаризовать «с нуля» из физики. В коде `mps_node.py` используется *другая*, каноническая форма $(s, v, \theta, \omega, e_{\text{int}})$. Концептуально это та же модель, просто:

- вместо двух декартовых (p_x, p_y) — одна продольная s (мы заранее знаем, что в сценарии $p_y \equiv 0$);
- плюс интегральный член e_{int} , чтобы устранить установившуюся ошибку скорости.

Конкретные матрицы $\mathbf{A}, \mathbf{B}$ для этой формы синтезируются в `matlab/main.m` и экспортируются в `config.yaml`, секция `mps.matrices.{A,B,C,D}`. Размер тот же — 5×5 и 5×2 , но числа другие. Период тоже *другой*: $T_s = 0.02$ с (50 Гц), а не 0.05 с легаси-контур.

Примечание

В коде эти матрицы приходят в *непрерывной* форме (контракт docs/mps/api.md). mps_node ZOH-дискретизирует их при загрузке/Apply — так что одна и та же $\mathbf{A}$ работает и в симе на ноуте, и на Pi, в любых обновлениях.

6.8 Что мы получили

- **Линеаризация:** ряд Тейлора 1-го порядка вокруг опорного $(v_0, 0)^T$. Получились $\mathbf{A} \in \mathbb{R}^{5 \times 5}$, $\mathbf{B} \in \mathbb{R}^{5 \times 2}$ с явными формулами (6.3), (6.4).
- Объект **полностью управляем** при $v_0 \neq 0$.
- **Дискретизация ZOH:** $\mathbf{A}_d = e^{\mathbf{A}T_s}$, $\mathbf{B}_d$ — через интеграл. Вычисляется одним вызовом ehm от блочной матрицы.
- Собственные значения непрерывной системы: три нуля (интеграторы по p_x, p_y, θ) и два отрицательных. Открытый объект не асимптотически устойчив, но управляем.
- Каноническая МПС-форма — та же модель с $(s, \cdot, \theta, \omega, e_{\text{int}})$ вместо $(p_x, p_y, \theta, v, \omega)$ и $T_s = 0.02$ с.

Теперь у нас есть всё, что нужно для регулятора. Дальше начинаем строить управляющий контур.

Chapter 7

Что такое регулятор

7.1 Вопрос

У нас есть модель $\mathbf{x}_{k+1} = \mathbf{A}_d \mathbf{x}_k + \mathbf{B}_d \mathbf{u}_k$. Робот «понимает», как состояние реагирует на команду. **Какую команду $\mathbf{u}_k$ ему подавать на каждом тике**, чтобы достичь цели?

Эта задача — задача синтеза *регулятора*.

Идея

Регулятор — это *функция*, которая по текущему состоянию (и, возможно, желаемому состоянию) выдаёт управление:

$$\mathbf{u}_k = \pi(\mathbf{x}_k, \mathbf{x}_k^{\text{ref}}).$$

Слово «регулятор» — из ТАУ. Часто говорят также «контроллер», «controller», «policy». Это одно и то же.

Идея этой главы — *интуитивно* объяснить, чем отличаются простой П-регулятор, PID и ЛКР. К концу главы будет понятно, *зачем* мы вообще идём в матрицы $\mathbf{Q}$, $\mathbf{R}$ и почему нельзя обойтись пропорциональным коэффициентом.

7.2 П-регулятор (proportional)

Самый простой регулятор: чем дальше от цели — тем сильнее тяни.

$$u_k = -k_p \cdot (x_k - x_k^{\text{ref}}). \quad (7.1)$$

Здесь $k_p > 0$ — *коэффициент усиления* (gain). Чем он больше, тем «жестче» регулятор. Скаляр; в многомерном случае — вектор/матрица.

Скаляр, удержание скорости

Хотим, чтобы робот ехал ровно v_{target} . Текущая скорость v . Тогда $u = -k_p(v - v_{\text{target}})$. Если $v < v_{\text{target}}$ — команда положительная, разгон. Если $v > v_{\text{target}}$ — отрицательная, торможение.

Проблемы П-регулятора:

- **Установившаяся ошибка.** Если есть сопротивление (трение, уклон), при достижении $v = v_{\text{target}}$ команда обнуляется, но трение остаётся — робот замедлится. На равновесии останется ошибка $\neq 0$.
- **Колебания.** Большой $k_p \rightarrow$ перерегулирование $\rightarrow$ качается вокруг цели.
- **Не учитывает динамику.** П-регулятор «слепой»: ему всё равно, что моторы инерционны. Если τ_v велико — регулятор будет «опаздывать» на каждом тике.

7.3 PID

Классическая попытка починить проблемы П: добавить *интеграл* ошибки (чтобы убрать установившуюся) и *производную* (чтобы предугадать).

$$u_k = -k_p \cdot e_k - k_i \cdot \sum_{j=0}^k e_j \Delta t - k_d \cdot (e_k - e_{k-1}) / \Delta t. \quad (7.2)$$

PID — старший товарищ. Работает на всём: от лифта до автопилота. Но у него тоже минусы:

- Подбор k_p, k_i, k_d — искусство (Зиглер–Никольс, ручное крутение). На многомерных задачах перестаёт быть удобным.
- Не оптимален по никакому критерию: «работает» $\neq$ «работает лучше всего».
- **Не знает модель** объекта. Если бы знал $\mathbf{A}_d, \mathbf{B}_d$ — мог бы предсказывать, а не реагировать. PID не предсказывает.

7.4 Линейно-квадратичный регулятор (ЛКР)

ЛКР (linear-quadratic regulator, LQR) — иначе подходит к задаче.

Идея

Идея ЛКР: вместо ручного подбора коэффициентов *сформулировать* цель как минимизацию специальной функции **критерия качества**, и пусть математика сама найдёт лучший регулятор для этой цели.

Критерий — сумма «штрафов» по всему будущему:

ЛКР-критерий качества

$$J = \sum_{k=0}^{\infty} \left(\mathbf{x}_k^T \mathbf{Q} \mathbf{x}_k + \mathbf{u}_k^T \mathbf{R} \mathbf{u}_k \right). \quad (7.3)$$

Здесь:

- $\mathbf{Q} \in \mathbb{R}^{n \times n}$ — штраф за **отклонения состояния** от нуля (предполагаем, что цель — $\mathbf{x} = 0$; если цель ненулевая, перейдём в координаты ошибки).
- $\mathbf{R} \in \mathbb{R}^{r \times r}$ — штраф за **величину управления**. Большой $\mathbf{R}$ — регулятор будет «экономить» моторы, малый — «топить педаль в пол».
- Обе матрицы — симметричные, положительно (полу-)определённые. На практике диагональные: $\mathbf{Q} = \text{diag}(q_1, \dots, q_n)$, $\mathbf{R} = \text{diag}(r_1, \dots, r_r)$.

7.4.1 Что значит «положительно определённая»

Примечание

Матрица $\mathbf{Q}$ **положительно (полу-)определённая**, если для любого ненулевого $\mathbf{x}$ выполняется $\mathbf{x}^T \mathbf{Q} \mathbf{x} > 0$ (или ≥ 0 — полу-).

Для диагональной $\mathbf{Q} = \text{diag}(q_1, \dots, q_n)$ это сводится к $q_i \geq 0$ (или строгая положительность для всех q_i , тогда уже positive definite).

Зачем требование: штраф $\mathbf{x}^T \mathbf{Q} \mathbf{x}$ должен быть *неотрицательным*, иначе оптимизатор «разгонит» состояние до $-\infty$ и получит бесконечную выгоду. Аналогично $\mathbf{R}$ должна быть строго положительно определённая — иначе можно бесконечно увеличивать $\mathbf{u}$ без штрафа.

7.4.2 Зачем квадратичный штраф?

Почему именно $\mathbf{x}^T \mathbf{Q} \mathbf{x}$, а не, скажем, $|\mathbf{x}|$?

- Дифференцируем $\rightarrow$ можно решить аналитически.
- Маленькие ошибки штрафуются *ещё меньше* (квадрат малого числа — ещё меньше), большие — *гораздо больше* (квадрат 5 = 25, квадрат 10 = 100). Регулятор сильно реагирует на большие отклонения, мягко — на мелкие.
- Удобная аналогия с энергией / расстоянием. Норма $\|\mathbf{x}\|^2 = \mathbf{x}^T \mathbf{x}$, а $\mathbf{x}^T \mathbf{Q} \mathbf{x}$ — «взвешенная» норма.

7.4.3 Результат ЛКР

Ключевой факт. Если объект линейный, дискретный, управляемый, и критерий (7.3) с положительно определёнными $\mathbf{Q}, \mathbf{R}$, то оптимальный регулятор — **линейная обратная связь**:

ЛКР-закон управления

$$\mathbf{u}_k = -\mathbf{K} \mathbf{x}_k, \quad (7.4)$$

где $\mathbf{K}$ — матрица $r \times n$, не зависящая от k (постоянная). Она вычисляется *один раз*, оффлайн, через *уравнение Риккати*. Как именно — следующая глава.

7.5 Интуитивная карта

Регулятор	Идея
П	«Тяни сильнее, если дальше».
PID	П + интеграл + производная (борьба с трением и инерцией).
ЛКР	Минимизируй квадратичный критерий $J = \sum (\mathbf{x}^\top \mathbf{Q} \mathbf{x} + \mathbf{u}^\top \mathbf{R} \mathbf{u})$. Решение — $\mathbf{u} = -\mathbf{K} \mathbf{x}$. Оптimalен по выбранному критерию.
МРС	Каждый тик решай ЛКР-подобную задачу на короткий горизонт вперёд, с учётом ограничений $\mathbf{u}_{\min} \leq \mathbf{u} \leq \mathbf{u}_{\max}$.

Идея

Логическая цепочка:

- П — слепая реакция.
- PID — реакция с памятью и предсказанием в эвристике.
- ЛКР — *оптимальная* реакция при заданном критерии и *знании модели*. Не учитывает ограничения.
- МРС — ЛКР с учётом ограничений на $\mathbf{u}$ и предсказанием на конечный горизонт N .

Каждый следующий — более мощный и при этом более вычислительно дорогой. В проекте используются ЛКР и МРС одновременно (МРС при `solver=clip` сводится к ЛКР с клиппингом — эффективный компромисс).

7.6 Связь с проектом

В проекте есть оба:

- `pi_nodes/control/lqr_controller.py` — чистый ЛКР (используется как baseline и для сравнения).
- `pi_nodes/control/mpc_controller.py` — МРС. В режиме `solver = "clip"` он *буквально* вычисляет ЛКР-усиление $\mathbf{K}_{\text{first}}$ (первая строка лифтированной QR-задачи), применяет его и клиппит в ограничения.

В следующих двух главах разберём ЛКР, потом МРС.

Chapter 8

ЛКР: оптимальный регулятор

8.1 Вопрос

В прошлой главе мы сказали: ЛКР даёт $\mathbf{u}_k = -\mathbf{K}\mathbf{x}_k$, и $\mathbf{K}$ считается через *уравнение Риккати*. Сейчас аккуратно разберём, *что* это за уравнение и *откуда* оно берётся.

Идея

ЛКР — это *задача оптимизации* на бесконечном горизонте. Решить её «в лоб» (например, методом Беллмана) можно, но получается тяжёлая рекурсия. Чудо в том, что для **линейного объекта с квадратичным критерием** эта рекурсия сходится к одной матрице $\mathbf{P}$, которую можно найти из **алгебраического уравнения**.

Это уравнение — DARE (Discrete-time Algebraic Riccati Equation).

8.2 Постановка задачи

Объект:

$$\mathbf{x}_{k+1} = \mathbf{A}_d \mathbf{x}_k + \mathbf{B}_d \mathbf{u}_k, \quad \mathbf{x}_0 \text{ задано.}$$

Критерий:

$$J = \sum_{k=0}^{\infty} \left(\mathbf{x}_k^T \mathbf{Q} \mathbf{x}_k + \mathbf{u}_k^T \mathbf{R} \mathbf{u}_k \right).$$

Цель: найти последовательность $\{\mathbf{u}_k\}_{k=0}^{\infty}$, которая **минимизирует** J .

8.3 Принцип динамического программирования

Идея

Принцип Беллмана. Оптимальная политика на бесконечном горизонте обладает свойством: *какова бы ни была начальная ситуация и первое решение, последующие решения должны быть оптимальными для оставшейся задачи.*

Для квадратичного критерия и линейной динамики из этого принципа следует, что **функция ценности** (стоимость до бесконечности из состояния $\mathbf{x}$) тоже квадратичная:

$$V^*(\mathbf{x}) = \mathbf{x}^\top \mathbf{P} \mathbf{x},$$

где $\mathbf{P}$ — симметричная положительно определённая матрица.

Подставляем в уравнение Беллмана:

$$\mathbf{x}^\top \mathbf{P} \mathbf{x} = \min_{\mathbf{u}} \left\{ \mathbf{x}^\top \mathbf{Q} \mathbf{x} + \mathbf{u}^\top \mathbf{R} \mathbf{u} + (\mathbf{A}_d \mathbf{x} + \mathbf{B}_d \mathbf{u})^\top \mathbf{P} (\mathbf{A}_d \mathbf{x} + \mathbf{B}_d \mathbf{u}) \right\}.$$

Минимум по $\mathbf{u}$ ищется аналитически: производная по $\mathbf{u}$, равная нулю.

8.3.1 Шаг 1: найти оптимальное $\mathbf{u}^*$

$$\begin{aligned} \frac{\partial}{\partial \mathbf{u}} \left[\mathbf{u}^\top \mathbf{R} \mathbf{u} + (\mathbf{A}_d \mathbf{x} + \mathbf{B}_d \mathbf{u})^\top \mathbf{P} (\mathbf{A}_d \mathbf{x} + \mathbf{B}_d \mathbf{u}) \right] &= 2\mathbf{R} \mathbf{u} + 2\mathbf{B}_d^\top \mathbf{P} (\mathbf{A}_d \mathbf{x} + \mathbf{B}_d \mathbf{u}) \\ &= 0. \end{aligned}$$

Раскрываем:

$$(\mathbf{R} + \mathbf{B}_d^\top \mathbf{P} \mathbf{B}_d) \mathbf{u} = -\mathbf{B}_d^\top \mathbf{P} \mathbf{A}_d \mathbf{x}.$$

Оптимальное управление через $\mathbf{P}$

$$\mathbf{u}^* = -\mathbf{K} \mathbf{x}, \quad \mathbf{K} = (\mathbf{R} + \mathbf{B}_d^\top \mathbf{P} \mathbf{B}_d)^{-1} \mathbf{B}_d^\top \mathbf{P} \mathbf{A}_d. \quad (8.1)$$

Это **линейный** закон: оптимальное управление — линейная функция текущего состояния. Чудо ЛКР именно в этом.

8.3.2 Шаг 2: уравнение для $\mathbf{P}$ (DARE)

Подставим $\mathbf{u}^* = -\mathbf{K} \mathbf{x}$ обратно в уравнение Беллмана и упростим. После выкладок получим:

DARE — Discrete Algebraic Riccati Equation

$$\mathbf{P} = \mathbf{A}_d^\top \mathbf{P} \mathbf{A}_d - \mathbf{A}_d^\top \mathbf{P} \mathbf{B}_d (\mathbf{R} + \mathbf{B}_d^\top \mathbf{P} \mathbf{B}_d)^{-1} \mathbf{B}_d^\top \mathbf{P} \mathbf{A}_d + \mathbf{Q}. \quad (8.2)$$

Это *нелинейное матричное уравнение* относительно $\mathbf{P}$.

8.4 Как решают DARE численно

Примечание

Аналитически решить DARE невозможно (кроме скалярных случаев). На практике используют численные методы:

- **Итерации Риккати** — запускаем рекурсию $P_{k+1} = \text{RHS DARE}(P_k)$ от любой начальной P_0 (часто Q). Сходится к решению, если объект управляем и Q, R положительны.
- **Метод Шура** (Шурава декомпозиция гамильтоновой матрицы) — стандарт в MATLAB и SciPy.

В нашем проекте — наивные итерации (см. `pi_nodes/control/_linalg.py`), fallback на `scipy.linalg.solve_discrete_are`, когда `scipy` есть.

`pi_nodes/control/_linalg.py`, функция `solve_discrete_are`

```

1 def solve_discrete_are(A, B, Q, R, max_iter=10_000, tol=1e-9):
2     """Решает DARE итерациями Беллмана."""
3     P = Q.copy()
4     for _ in range(max_iter):
5         BtP = B.T @ P
6         K = np.linalg.solve(R + BtP @ B, BtP @ A)
7         P_new = A.T @ P @ A - A.T @ P @ B @ K + Q
8         if np.max(np.abs(P_new - P)) < tol:
9             return P_new
10        P = P_new
11    raise RuntimeError("DARE did not converge")

```

После того как DARE решено и P найдена, считаем K по (8.1). Готово — регулятор готов к работе.

8.5 Какие Q, R выбрать

Идея

Выбор Q, R — это *инженерное искусство*. ЛКР не говорит, какие они должны быть; он только обещает *оптимальность* для тех, которые мы зададим.

Практические рецепты:

8.5.1 Правило Брайсона

Для каждой компоненты i положить $q_i = 1/x_{i,\max}^2$, где $x_{i,\max}$ — максимально *допустимое* отклонение. Аналогично для $r_j = 1/u_{j,\max}^2$. Это нормализует масштабы — регулятор не будет «думать», что отклонение в радианах (~ 0.1) важнее, чем в метрах (~ 5).

8.5.2 Дефолты проекта

config.yaml, секция control.weights

```
control:
  weights:
    Q_diag: [10.0, 10.0, 5.0, 1.0, 1.0]
    R_diag: [1.0, 1.0]
```

Вес	Компонента	Смысл
10	p_x	сильный штраф за «недокат» / «перекат»
10	p_y	сильный штраф за «съезд вбок»
5	θ	средний штраф за кривой курс
1	v	лёгкий штраф за «не та скорость»
1	ω	лёгкий штраф за «не то вращение»
1	u_v	вес на расход линейного управления
1	u_ω	вес на расход углового управления

8.5.3 Что меняется, если...

Что если?

$\mathbf{Q} \gg \mathbf{R}$ — регулятор готов «топить педаль в пол», лишь бы $\mathbf{x}$ был у цели. Быстро, но энергоёмко и с перерегулированием.

$\mathbf{Q} \ll \mathbf{R}$ — регулятор «бережёт моторы», медленно подходит к цели. Гладко, но долго.

$\mathbf{Q}$ не-диагональная — разрешает «обмен» между компонентами: можно сильно штрафовать, например, разницу $p_x - s_{\text{ref}}$, а не каждую компоненту отдельно. Усложняет интерпретацию, редко нужно.

$q_\theta = 0$ — регулятору всё равно, куда смотрит робот. Странно, но математически валидно: ЛКР решит DARE и выдаст $\mathbf{K}$, где курс будет «болтаться» свободно.

$r_i = 0$ — неуправляемо, DARE не сходится, $\mathbf{P}$ вырождается. *Не делать так.*

8.6 Замкнутая система

Идея

Подставим закон $\mathbf{u}_k = -\mathbf{K}\mathbf{x}_k$ в исходную модель:

$$\mathbf{x}_{k+1} = \mathbf{A}_d\mathbf{x}_k + \mathbf{B}_d(-\mathbf{K}\mathbf{x}_k) = (\mathbf{A}_d - \mathbf{B}_d\mathbf{K})\mathbf{x}_k.$$

Это **замкнутая система**. Её собственные значения $\lambda(\mathbf{A}_d - \mathbf{B}_d\mathbf{K})$ — *полюса* замкнутого контура. ЛКР гарантирует, что все они лежат внутри единичного круга ($|\lambda| < 1$) — замкнутая система *устойчива*.

Проверка в коде:

pi_nodes/control/mpc_controller.py:332

```

1 def closed_loop_eigenvalues(self) -> np.ndarray:
2     """ $|\lambda| < 1 \Leftrightarrow \text{closed-loop } \mathbf{x}_{k+1} = (\mathbf{A}_d - \mathbf{B}_d \mathbf{K}) \mathbf{x}$  is stable."""
3     return np.linalg.eigvals(self.Ad - self.Bd @ self.K_first)
4
5 def is_stable(self) -> bool:
6     return bool(np.all(np.abs(self.closed_loop_eigenvalues()) <
7                        1.0 - 1e-9))

```

8.7 Слабое место: ограничения

ЛКР **не знает** о том, что $u_v \leq 0.30$ м/с и $|u_\omega| \leq 2$ рад/с. Если задать $\mathbf{x}_0$ далеко от цели, закон $-\mathbf{K}\mathbf{x}_0$ выдаст $\mathbf{u}$, который физически невозможен.

Внимание

В коде `LQRController.step()` это решают *пост-обработкой*: после $-\mathbf{K}\mathbf{x}$ результат прогоняется через `np.clip(u, u_min, u_max)`. Это работает, но *не оптимально* — ЛКР проектировал под полную свободу. Когда $\mathbf{u}$ упирается в насыщение, его поведение перестаёт быть оптимальным. Это — *мотивация для MPC*. MPC формулирует ограничения внутри оптимизации, а не накладывает их сверху.

8.8 Что мы получили

- ЛКР — линейный оптимальный регулятор: $\mathbf{u}_k = -\mathbf{K}\mathbf{x}_k$.
- Матрица $\mathbf{K}$ считается *оффлайн* через DARE (8.2) — одно матричное уравнение, решается итерациями либо Шуром.
- ЛКР гарантирует устойчивость замкнутой системы (все полюса внутри единичного круга).
- Выбор $\mathbf{Q}, \mathbf{R}$ — инженерное искусство: правило Брайсона, диагональные веса.
- ЛКР **не знает** о box-ограничениях на $\mathbf{u}$. Клип сверху — костыль, мотивация для MPC.

Идём дальше: MPC.

Chapter 9

MPC: Model Predictive Control

9.1 Вопрос

ЛКР минимизирует J на *бесконечном* горизонте, выдаёт постоянное усиление $\mathbf{K}$ и не знает о ограничениях $\mathbf{u}_{\min} \leq \mathbf{u} \leq \mathbf{u}_{\max}$. Эти два недостатка устраняет MPC.

Идея

Идея MPC: вместо одного решения на ∞ — решать *маленькую* оптимизационную задачу *на каждом тике*, на конечный горизонт N вперёд, с явными ограничениями.

Receding horizon (скользящий горизонт):

1. Сейчас, в момент k , посмотри на N шагов вперёд.
2. Сосчитай *весь будущий план* $\mathbf{u}_0, \mathbf{u}_1, \dots, \mathbf{u}_{N-1}$, минимизируя стоимость.
3. Примени *только первый шаг* $\mathbf{u}_0$ к роботу.
4. На следующем тике начни заново с $\mathbf{x}_{k+1}$.

Это похоже на шахматы: считаем дерево на N ходов, делаем один, считаем заново.

9.2 Постановка

MPC-задача (с ограничениями)

$$\begin{aligned} \min_{\mathbf{u}_0, \dots, \mathbf{u}_{N-1}} J &= \sum_{i=0}^{N-1} \left(\mathbf{x}_i^T \mathbf{Q} \mathbf{x}_i + \mathbf{u}_i^T \mathbf{R} \mathbf{u}_i \right) + \mathbf{x}_N^T \mathbf{P}_f \mathbf{x}_N \\ \text{при условиях } \mathbf{x}_{i+1} &= \mathbf{A}_d \mathbf{x}_i + \mathbf{B}_d \mathbf{u}_i, \quad i = 0, \dots, N-1, \\ \mathbf{u}_{\min} \leq \mathbf{u}_i &\leq \mathbf{u}_{\max}, \quad i = 0, \dots, N-1, \\ \mathbf{x}_0 &\text{ задано (измерено сейчас).} \end{aligned} \tag{9.1}$$

Новое:

- Сумма по *конечному* N (не до бесконечности).
- **Терминальный штраф** $\mathbf{P}_f$ — штраф за последнее состояние $\mathbf{x}_N$. Если выбрать

$\mathbf{P}_f$ как решение DARE для $(\mathbf{A}_d, \mathbf{B}_d, \mathbf{Q}, \mathbf{R})$, то конечный горизонт ведёт себя «как бесконечный» — сумма от N до ∞ заменяется аккуратно квадратичным выражением.

- Явные бок-ограничения на $\mathbf{u}_i$.

9.3 Лифтированная динамика

Переменными оптимизации в (9.1) являются $\mathbf{u}_0, \dots, \mathbf{u}_{N-1}$ — всего rN скаляров. Состояние $\mathbf{x}_i$ не независимая переменная: оно *зависит* от $\mathbf{u}_i$ через $\mathbf{x}_{i+1} = \mathbf{A}_d \mathbf{x}_i + \mathbf{B}_d \mathbf{u}_i$. Удобно это зависимое представление сделать явным.

Идея

Лифтинг (lifting). Разворачиваем рекурсию по динамике.

$$\begin{aligned}\mathbf{x}_1 &= \mathbf{A}_d \mathbf{x}_0 + \mathbf{B}_d \mathbf{u}_0, \\ \mathbf{x}_2 &= \mathbf{A}_d \mathbf{x}_1 + \mathbf{B}_d \mathbf{u}_1 = \mathbf{A}_d^2 \mathbf{x}_0 + \mathbf{A}_d \mathbf{B}_d \mathbf{u}_0 + \mathbf{B}_d \mathbf{u}_1, \\ \mathbf{x}_3 &= \mathbf{A}_d^3 \mathbf{x}_0 + \mathbf{A}_d^2 \mathbf{B}_d \mathbf{u}_0 + \mathbf{A}_d \mathbf{B}_d \mathbf{u}_1 + \mathbf{B}_d \mathbf{u}_2, \\ &\vdots\end{aligned}$$

Складываем все состояния в большой столбец $\mathbf{X}$ и все управления — в $\mathbf{U}$:

Введём блочные векторы:

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \vdots \\ \mathbf{x}_N \end{pmatrix} \in \mathbb{R}^{nN}, \quad \mathbf{U} = \begin{pmatrix} \mathbf{u}_0 \\ \mathbf{u}_1 \\ \vdots \\ \mathbf{u}_{N-1} \end{pmatrix} \in \mathbb{R}^{rN}.$$

Тогда вся динамика «помещается» в одно матричное равенство:

Лифтированная динамика

$$\mathbf{X} = \Phi \mathbf{x}_0 + \Gamma \mathbf{U}, \quad (9.2)$$

где

$$\Phi = \begin{pmatrix} \mathbf{A}_d \\ \mathbf{A}_d^2 \\ \vdots \\ \mathbf{A}_d^N \end{pmatrix} \in \mathbb{R}^{nN \times n}, \quad \Gamma = \begin{pmatrix} \mathbf{B}_d & 0 & \cdots & 0 \\ \mathbf{A}_d \mathbf{B}_d & \mathbf{B}_d & \cdots & 0 \\ \vdots & & \ddots & \vdots \\ \mathbf{A}_d^{N-1} \mathbf{B}_d & \mathbf{A}_d^{N-2} \mathbf{B}_d & \cdots & \mathbf{B}_d \end{pmatrix} \in \mathbb{R}^{nN \times rN}.$$

pi_nodes/control/mpc_controller.py:134

```
1 def _build_qp_matrices(self):
2     n, r, N = self.n, self.r, self.N
3     Phi = np.zeros((n * N, n))
4     Gamma = np.zeros((n * N, r * N))
```

```

5     Apow = np.eye(n)
6     for i in range(N):
7         Apow = self.Ad @ Apow                                # A^{i+1}
8         Phi[i*n:(i+1)*n, :] = Apow
9         for j in range(i + 1):
10            block = np.linalg.matrix_power(self.Ad, i - j) @ self.
                Bd
11            Gamma[i*n:(i+1)*n, j*r:(j+1)*r] = block
12 # ... дальше Q_bar, R_bar, H, f_mat

```

9.3.1 Стоимость в лифтированной форме

Соберём блочно-диагональные веса (последний блок — терминальный):

$$\bar{\mathbf{Q}} = \begin{pmatrix} \mathbf{Q} & & & \\ & \mathbf{Q} & & \\ & & \ddots & \\ & & & \mathbf{P}_f \end{pmatrix} \in \mathbb{R}^{nN \times nN}, \quad \bar{\mathbf{R}} = \mathbf{I}_N \otimes \mathbf{R} \in \mathbb{R}^{rN \times rN}.$$

Подставим (9.2) в стоимость J :

$$J(\mathbf{U}) = (\Phi \mathbf{x}_0 + \Gamma \mathbf{U})^\top \bar{\mathbf{Q}} (\Phi \mathbf{x}_0 + \Gamma \mathbf{U}) + \mathbf{U}^\top \bar{\mathbf{R}} \mathbf{U}.$$

Раскрываем и группируем по степеням $\mathbf{U}$:

Квадратичная форма стоимости

$$J(\mathbf{U}) = \frac{1}{2} \mathbf{U}^\top \mathbf{H} \mathbf{U} + \mathbf{f}(\mathbf{x}_0)^\top \mathbf{U} + \text{const}, \quad (9.3)$$

$$\mathbf{H} = 2(\Gamma^\top \bar{\mathbf{Q}} \Gamma + \bar{\mathbf{R}}), \quad \mathbf{f}(\mathbf{x}_0) = 2\Gamma^\top \bar{\mathbf{Q}} \Phi \mathbf{x}_0.$$

«const» — слагаемое, зависящее только от $\mathbf{x}_0$. Для оптимизации по $\mathbf{U}$ оно неважно.

9.4 Квадратичное программирование (QP)

С учётом ограничений MPC превращается в каноническую **QP-задачу**:

QP-постановка MPC

$$\min_{\mathbf{U}} \frac{1}{2} \mathbf{U}^\top \mathbf{H} \mathbf{U} + \mathbf{f}^\top \mathbf{U}, \quad \mathbf{U}_{\min} \leq \mathbf{U} \leq \mathbf{U}_{\max}. \quad (9.4)$$

Здесь $\mathbf{U}_{\min}, \mathbf{U}_{\max}$ — повторение $\mathbf{u}_{\min}, \mathbf{u}_{\max}$ по N блокам.

Свойства задачи:

- $\mathbf{H} = \mathbf{H}^\top$, положительно определённая $\Rightarrow$ задача **выпуклая**, имеет единственный минимум.

- Ограничения — линейные неравенства, выпуклые.
- Существуют десятки QP-солверов (OSQP, qpOASES, scipy.optimize и др.).

9.5 Два режима в проекте: clip и qp

В `pi_nodes/control/mpc_controller.py` реализованы две стратегии решения:

9.5.1 clip (по умолчанию)

Без ограничений (в задаче, не в железе) решение QP (9.4) аналитическое — производная по U , равная нулю:

$$\mathbf{H}U^* + \mathbf{f} = 0 \Rightarrow U^* = -\mathbf{H}^{-1}\mathbf{f}(x_0).$$

Берём первые r компонент — это $\mathbf{u}_0$. После лишних выкладок получается, что

$$\mathbf{u}_0 = -\mathbf{K}_{\text{first}} \mathbf{x}_0, \quad \mathbf{K}_{\text{first}} = (\mathbf{H}^{-1} 2\mathbf{\Gamma}^T \bar{\mathbf{Q}} \Phi)_{[1\dots r, :]}. \quad (9.5)$$

То есть аналитический MPC — это **та же линейная обратная связь**, что ЛКР, только усиление $\mathbf{K}_{\text{first}}$ другое (зависит от N). После умножения мы *клипим* управление к ограничениям:

$$\mathbf{u}_0 \leftarrow \text{clip}(\mathbf{u}_0, \mathbf{u}_{\min}, \mathbf{u}_{\max}).$$

Примечание

Когда это оптимально: ровно когда невозмущённое решение $-\mathbf{K}_{\text{first}} \mathbf{x}_0$ *u* так попадает в `box`. Тогда клип ничего не меняет. Если выходит за границы — клиппинг даёт *допустимое, но не оптимальное* управление. В большинстве рабочих режимов работа этого хватает.

`mpc_controller.py:288`

```

1 def step(self, x, x_ref=None):
2     if x_ref is None:
3         dx = x
4     else:
5         dx = x - x_ref
6     if self._solver == "qp":
7         return self._solve_qp(dx)
8     # Explicit clip
9     u = -self.K_first @ dx
10    return np.clip(u, self.u_min, self.u_max)

```

9.5.2 qp (полный)

При `control.mpc.solver: qp` в конфиге проект решает QP *честно*:

`mpc_controller.py:303`

```

1 def _solve_qp(self, dx):
2     f = self.f_mat @ dx
3     H = self.H
4     u_lo = np.tile(self.u_min, self.N)
5     u_hi = np.tile(self.u_max, self.N)
6     bounds = list(zip(u_lo, u_hi))
7     U0 = -np.linalg.solve(H, f)           # warm start
8     U0 = np.clip(U0, u_lo, u_hi)
9     res = minimize(lambda U: 0.5*U@H@U + f@U,
10                    U0, jac=lambda U: H@U + f,
11                    method="L-BFGS-B", bounds=bounds)
12     return (res.x if res.success else U0)[:self.r]
```

Используется L-BFGS-B (scipy) с warm start от аналитического решения. Делает то же, что OSQP, только медленнее. Включают только если `clip` регулярно упирается в ограничения.

9.6 Выбор горизонта N

Идея

Большой N (например, 50):

- Регулятор «видит дальше» — может заранее тормозить, чтобы не перелететь цель.
- QP становится больше: размер U растёт как rN , размер H — как $(rN)^2$. Время на тик растёт нелинейно.

Маленький N (например, 1):

- Решается быстро.
- Регулятор «близорукий», поведение ближе к ЛКР с малым весом конечного штрафа.

Рабочий компромисс: $N \cdot T_s \gtrsim 2\tau_{\text{slow}}$, где τ_{slow} — самая медленная постоянная времени объекта. Для нас $\tau_{\text{slow}} = \tau_v = 0.15$ с, $T_s = 0.02$ с $\Rightarrow N \geq 15$. В дефолтах — $N = 10$ или 20 (см. `mps.horizon_N`).

9.7 Терминальный штраф P_f

Идея

Проблема конечного горизонта: что мешает MPC сделать «хитрое» движение, которое в окне $[0, N]$ выглядит выгодно, а после N оставляет робот «в плохой позе» (далеко от цели)? Ничего, если стоимость после N мы игнорируем.

Решение: добавить терминальный штраф $\mathbf{x}_N^T \mathbf{P}_f \mathbf{x}_N$, который аппроксимирует «стоимость до бесконечности» из $\mathbf{x}_N$.

Лучшая такая аппроксимация — решение DARE (8.2) для $(\mathbf{A}_d, \mathbf{B}_d, \mathbf{Q}, \mathbf{R})$, то есть $\mathbf{P}_f = \mathbf{P}_{\text{LQR}}$. Тогда MPC в пределе $N \rightarrow \infty$ сходится к ЛКР, а на конечном N ведёт себя «как ЛКР, но с уважением к ограничениям».

mpc_controller.py:126

```
1 def _compute_terminal_penalty(self):
2     """If Pf not given, solve DARE for a guaranteed-stable
3         terminal cost."""
4     return solve_discrete_are(self.Ad, self.Bd, self.Q, self.R)
```

9.8 Слежение за reference

До сих пор минимизировали $\mathbf{x}^T \mathbf{Q} \mathbf{x}$ — штраф за отклонение от нуля. Но цель сценария — $\mathbf{x}^{\text{ref}} \neq 0$ (например, s_{ref} растёт линейно к D). Стандартный приём — работать в ошибке слежения $\Delta \mathbf{x} = \mathbf{x} - \mathbf{x}^{\text{ref}}$:

$$J = \sum_{i=0}^{N-1} \left((\mathbf{x}_i - \mathbf{x}^{\text{ref}})^T \mathbf{Q} (\mathbf{x}_i - \mathbf{x}^{\text{ref}}) + \mathbf{u}_i^T \mathbf{R} \mathbf{u}_i \right) + (\mathbf{x}_N - \mathbf{x}^{\text{ref}})^T \mathbf{P}_f (\mathbf{x}_N - \mathbf{x}^{\text{ref}}). \quad (9.6)$$

В коде — именно так:

mpc_controller.py:288

```
1 def step(self, x, x_ref=None):
2     dx = x - x_ref if x_ref is not None else x
3     ...
```

9.9 Sanity-check замкнутой системы

После rebuild() (Apply из UI) можно убедиться, что новый регулятор устойчив:

mpc_controller.py:332

```
1 def closed_loop_eigenvalues(self):
2     """|λ| < 1 ⇔ closed-loop x_{k+1} = (Ad - Bd K_first) x is
3         stable."""
4     return np.linalg.eigvals(self.Ad - self.Bd @ self.K_first)
5
6 def is_stable(self):
7     return bool(np.all(np.abs(self.closed_loop_eigenvalues()) <
8         1.0 - 1e-9))
```

Этот тест в UI делает «Validate matrices» — кнопка перед Apply.

9.10 Hot-reload между прогонами

В МПС-учебном модуле пользователь *редактирует* матрицы из UI и жмёт Apply. На Pi прилетает mps/matrices/set, и нода вызывает mpc.rebuild(): атомарно пересчитываются $\Phi, \Gamma, \mathbf{H}, \mathbf{K}_{\text{first}}$, при ошибке — rollback к предыдущему снимку. Это и есть «учебный пульт».

mpc_controller.py:172, фрагмент rebuild

```

1 def rebuild(self, *, Ad=None, Bd=None, Q_diag=None, R_diag=None,
2             N=None, u_min=None, u_max=None, Pf=None):
3     snapshot = (self.Ad.copy(), ..., self.K_first.copy())
4     try:
5         # обновить поля, провалидировать формы, пересчитать
6         # matrices
7         self._build_qp_matrices()
8     except Exception:
9         # Rollback on ANY failure
10        (self.Ad, ..., self.K_first) = snapshot
11        raise

```

9.11 Что мы получили

- МРС — решение QP-задачи на каждом тике с явными ограничениями.
- Через *лифтинг* динамики QP сводится к канонической форме $\frac{1}{2}\mathbf{U}^T\mathbf{H}\mathbf{U} + \mathbf{f}^T\mathbf{U}$.
- Без ограничений решение аналитическое, $\mathbf{u}_0 = -\mathbf{K}_{\text{first}}\mathbf{x}_0$. С ограничениями — настоящая QP-задача, в проекте — L-BFGS-B с warm start.
- Дефолтный режим — clip: считаем $\mathbf{K}_{\text{first}}$, умножаем, клипуем. Быстро и в большинстве случаев оптимально.
- Терминальный штраф $\mathbf{P}_f$ — решение DARE; делает короткий горизонт «бесконечно-устойчивым».
- Горизонт N — компромисс: $NT_s \gtrsim 2\tau_{\text{slow}}$.
- rebuild() — атомарное пересоздание контроллера с rollback на ошибку; используется при Apply из UI.

Теперь, наконец, всё готово, чтобы запустить настоящий сценарий.

Chapter 10

Сценарий «D метров вперёд»

10.1 Что мы строим

Соберём всё: модель $(\mathbf{A}_d, \mathbf{B}_d)$, регулятор ($\mathbf{K}_{\text{first}}$ из MPC), измерение состояния $(s, v, \theta, \omega, e_{\text{int}})$, оркестратор (FSM). Получится **сценарий** — от нажатия кнопки в UI до полного отчёта.

Идея

Сценарий — это «контракт» между UI и роботом. Он говорит: *при каких параметрах робот считается успешно справившимся, а при каких — нет*. В MPC-модуле сценарий один: «проехать D м вперёд» (с опциональным предварительным разворотом, о нём — глава 11). Каждый прогон — финитный, не цикл.

10.2 Постановка задачи

Дано:

- Дистанция $D \in (0, D_{\text{max}}]$ м, по умолчанию 2.0.
- Целевая скорость $v_{\text{target}} \in (0, v_{\text{max}}]$ м/с, по умолчанию 0.15.
- Целевой курс $\varphi \in [-\pi, \pi]$ (см. главу 12). По умолчанию 0 — едем прямо.

Требуется: от старта (положение s_{start} , курс θ_{start}) переместиться в точку с относительной координатой $s = D$, курс удерживать у φ .

Допустимые исходы:

Событие	Условие	status
Достижение цели	$s \geq D - 0.05$ м	reached
Тайм-аут	$t > 3 D/v_{\text{target}}$	timeout
Принудит. abort	MQTT mps/scenario/abort	aborted
Расходимость	$\ \mathbf{x}\ > \text{bound, NaN/Inf}$	error

10.3 Reference

МРС требует $\mathbf{x}_k^{\text{ref}}$ — куда хотим попасть на каждом шаге. Самая простая стратегия: **линейный разгон до D за номинальное время.**

Reference сценария

$$s_{\text{ref}}(k) = \min(D, k \Delta t \cdot v_{\text{target}}), \quad \mathbf{x}_k^{\text{ref}} = \begin{pmatrix} s_{\text{ref}}(k) \\ v_{\text{target}} \\ \varphi \\ 0 \\ 0 \end{pmatrix}. \quad (10.1)$$

Геометрия: s_{ref} растёт линейно с угловым коэффициентом v_{target} , пока не достигнет D — дальше «плато». Курс $\theta_{\text{ref}} = \varphi$ постоянен. Угловая скорость $\omega_{\text{ref}} = 0$ — робот не должен крутиться, удерживая курс.

Примечание

Почему $v_{\text{ref}} = v_{\text{target}}$ *константа*, а не «по правилу»? Потому что в постоянной части s_{ref} (после достижения плато) скорость и так логически 0, но **штраф Q по v должен быть маленький** — иначе МРС будет «тормозить» заранее, не доезжая до D . В `config.yaml` мы видим $Q_{\text{diag}}[v] = 1$ против $Q_{\text{diag}}[s] = 10$ — скорость важна сильно меньше позиции.

10.4 Полный тик регулятора

Один полный шаг `mps_node` (50 Гц):

1. **Прочитать измерение.** Берём последний `x_meas` из одометрии. Конвертируем в относительные координаты (4.3).
2. **Построить reference.** По текущему k собираем $\mathbf{x}_k^{\text{ref}}$ из (10.1).
3. **Шаг МРС.** `u = mpc.step(x_meas, x_ref)`. Под капотом — $\mathbf{u}_0 = -\mathbf{K}_{\text{first}}(\mathbf{x} - \mathbf{x}^{\text{ref}}) + \text{клиппинг}$.
4. **Sanity-check.** Если в $\mathbf{u}$ или $\mathbf{x}$ есть NaN/Inf — немедленный `status='error'`.
5. **Жёсткий сар угловой скорости в forward-режиме.** $|u_{\omega}| \leq 0.5$ рад/с (а не 2.0 как глобально) — защита от случайного «винта» при удержании курса.
6. **Опубликовать.** `cmd_vel` в MQTT (`linear_x`, `angular_z`); телеметрию `x`, `u`, `y` — в UI.
7. **Проверить условия завершения.** Достигли? Тайм-аут? Watchdog (нет одометрии 3 тика подряд)?

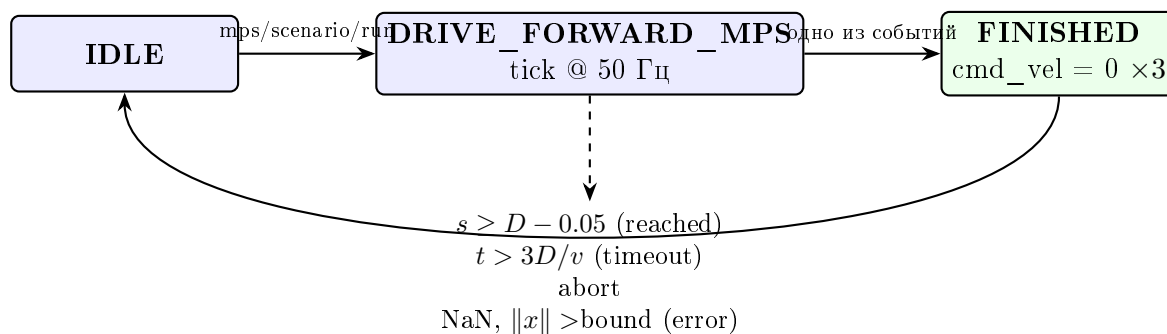
pi_nodes/nodes/mps_node.py:436

```

1  def _tick_drive(self, run, x):
2      phi = run.target_heading
3      # Reference: ramp s_ref to D, hold v_target, hold heading  $\varphi$ .
4      s_ref = min(run.distance, run.drive_t * run.v_target)
5      x_ref = np.array([s_ref, run.v_target, phi, 0.0, 0.0])
6
7      try:
8          u = self._mpc.step(x, x_ref=x_ref)
9      except Exception as exc:
10         self._finish_run('error', f'mpc.step: {exc}')
11         return
12
13     if not (np.all(np.isfinite(u)) and np.all(np.isfinite(x))):
14         self._finish_run('error', 'NaN/Inf in u or x')
15         return
16
17     # Hard omega cap in forward scenario.
18     u[1] = max(-self._omega_max_fwd,
19               min(self._omega_max_fwd, u[1]))
20
21     self._publish_cmd_and_telemetry(run, x, u)
22     # ... check reached, timeout, watchdog

```

10.5 Lifecycle прогона



Failsafe в FINISHED: моторам отправляется команда $[0, 0]$ **три раза** подряд. Это страховка на случай, если первый пакет потеряется в MQTT — мотор-нода имеет встроенный watchdog `cmd_vel_timeout`, который сам стопнет двигатели, но дублирование не помешает.

10.6 Метрики качества

После окончания прогона на основе массива телеметрии считаются **метрики**. Они отвечают на вопрос «как хорошо отработал регулятор». Без них «работает» не значит «хорошо».

Метрика	Формула	Что значит
overshoot	$\max_t s(t) - D$, обрезано ≥ 0	«Пролетели мимо»
settling_time	первое t , после которого $ s - D < 0.02$ м	«Сколько успокаивались»
control_energy	$\sum_k \mathbf{u}_k^T \mathbf{R} \mathbf{u}_k \Delta t$	«Расход управления»
ss_error	$ s(t_{\text{end}}) - D $	«Установившаяся ошибка»
peak_v	$\max_t v(t) $	«Пиковая скорость»
peak_omega	$\max_t \omega(t) $	«Пиковая угловая скорость»

Идея

Учебный смысл метрик. Можно специально подобрать «плохой» $\mathbf{Q}$ или $\mathbf{R}$ так, чтобы:

- получить большой overshoot (например, r_v малая — регулятор «топит педаль»);
- получить большой settling_time (r_v большая — регулятор «медленный»);
- получить большую ss_error (q_s малая — регулятору не критично, доехал ли).

Это и есть «учебный пульт» из спеки: пробуем разные веса, смотрим, что получилось. Курсовая.

10.7 Safety

Чтобы случайный «учебный пульт» не сломал железо, есть многоуровневая защита:

1. **Pre-validate:** запрос с $D > D_{\max}$ или $v_{\text{target}} > v_{\max}$ отбрасывается ещё на REST (HTTP 400) и ещё раз на Pi (mps/error, error_type="precondition").
2. **Watchdog по одометрии:** если 3 тика подряд нет odom — прогон завершается ошибкой watchdog.
3. **FSM-lock:** в DRIVE_FORWARD_MPS игнорируются voice/joystick/детекции мяча. Только abort переключает состояние.
4. **Жёсткий cap u_ω :** в forward-режиме $|u_\omega| \leq 0.5$ рад/с (это меньше, чем общий $u_{\max}[1] = 2$).
5. **Failsafe:** при любом FINISHED тройной cmd_vel = 0.

10.8 Симулятор

Параллельно с роботом существует *идеальный* симулятор — compute_node/mps_runner.py. Он гоняет ровно ту же последовательность mps.step(x, x_ref), но вместо одометрии обновляет $\mathbf{x}$ по дискретной модели:

$$\mathbf{x}_{k+1} = \mathbf{A}_d \mathbf{x}_k + \mathbf{B}_d \mathbf{u}_k.$$

Без шумов, без motor lag, без проскальзывания. Если запустить «Run on Sim» из UI, прогон выполнится за ~ 100 мс прямо на бэкенде — никакого железа. Это нужно:

- для UI — быстрый итеративный подбор $\mathbf{Q}, \mathbf{R}$ без возни с роботом;
- для тестов — юнит-тесты не требуют железа;
- для сравнения — разница между `sim` и реальным прогоном показывает, насколько модель отстаёт от реальности.

10.9 Что мы получили

- Сценарий определяет, *что значит* «успешный прогон»: `reached`, `timeout`, `aborted`, `error`.
- Reference — линейный разгон s_{ref} до D за номинальное время.
- Один тик — 7 шагов от чтения одометрии до публикации `cmd_vel`.
- Метрики — объективная мера качества, считаются по концу прогона.
- Безопасность — 5 уровней защиты от ломки железа учебным пультом.
- Симулятор — идеальная модель, быстрая проверка без робота.

Это была *одно*фазная картина. Но в текущей ветке мы добавили **предварительный разворот** к выбранному курсу. Этим и займёмся.

Chapter 11

Двухфазный сценарий: TURN, затем DRIVE

11.1 Вопрос

В прошлой главе сценарий ехал прямо «вперёд от текущей ориентации». А если пользователь *выбрал* цель не прямо перед роботом, а под углом φ (например, $+45^\circ$ влево)? Что делать?

Идея

Можно было бы дать MPC сразу полную задачу: «доехать до точки $(D \cos \varphi, D \sin \varphi)$, развернувшись по ходу». Но это плохая идея для линеаризованной вокруг *прямолинейного* движения модели: $\varphi = 45^\circ$ — далеко не «малое отклонение», линейная модель там врёт.

Решение — двухфазный сценарий:

1. **TURN.** Робот стоит на месте, поворачивается к курсу φ . После завершения — курс точно φ , позиция не изменилась.
2. **DRIVE.** Робот едет D метров, удерживая курс φ . Это уже та же задача из главы 10, только $\theta_{\text{ref}} = \varphi$ вместо 0.

Между фазами — мгновенный переход (без паузы), но с *разными ограничениями* на u_ω и *другим reference*.

11.2 Фаза TURN

11.2.1 Reference

В фазе TURN мы хотим, чтобы робот стоял ($s = 0, v = 0$) и развернулся к курсу φ :

Reference фазы TURN

$$\mathbf{x}_{\text{turn}}^{\text{ref}} = \begin{pmatrix} 0 \\ 0 \\ \varphi \\ 0 \\ 0 \end{pmatrix}. \quad (11.1)$$

Подадим этот reference в `mps.step(x, x_ref)`. Полученное $\mathbf{u}_{\text{mps}}$ будет «оптимальным» — но MPC не знает, что мы хотим *чистый разворот*, поэтому без дополнительных охранных u_v окажется не нулевым (особенно если φ большой).

11.2.2 Принудительные ограничения фазы

В коде это решают **пост-обработкой**:

pi_nodes/nodes/mps_node.py:384, фаза TURN

```

1  def _tick_turn(self, run, x):
2      phi = run.target_heading
3
4      # Курс совпал с целью → переход в DRIVE.
5      if abs(_normalize_angle(x[_THETA] - phi)) < self._turn_tol:
6          run.phase = 'drive'
7          run.drive_t = 0.0
8          return True
9
10     x_ref = np.array([0.0, 0.0, phi, 0.0, 0.0])
11     u = self._mps.step(x, x_ref=x_ref)
12
13     # Чистое вращение: ход — в ноль;  $\omega$  — свой (более высокий) кап.
14     u[0] = 0.0
15     u[1] = max(-self._omega_max_turn,
16               min(self._omega_max_turn, u[1]))
17
18     self._publish_cmd_and_telemetry(run, x, u)
19     # ... watchdog, timeout

```

Что происходит:

1. $u[0] = 0.0$ — принудительный ход в ноль. MPC мог захотеть двигаться, но мы не разрешаем.
2. $u[1]$ — угловая скорость зажимается в $[-\omega_{\text{max,turn}}, +\omega_{\text{max,turn}}]$, где $\omega_{\text{max,turn}} = 1.0$ рад/с — *больше*, чем в DRIVE-фазе (0.5). На месте можно крутиться быстрее.

Примечание

Почему *разный* сар на ω в TURN и DRIVE? Потому что:

- В TURN мы хотим быстро развернуться — иначе зачем вообще фаза.
- В DRIVE *большой* ω при движении — это «винт», робот съезжает с курса.

Лучше держать узкий сар, чтобы регулятор был «нежным» в коррекциях.

Два параметра: `mps.scenario.omega_max_in_turn = 1.0` и `omega_max_in_forward = 0.5`.

11.2.3 Условие завершения TURN

Переходим в DRIVE, когда курс совпал с целью с заданным допуском:

$$|\text{wrap}_{[-\pi, \pi]}(\theta - \varphi)| < \varepsilon_{\text{turn}}. \quad (11.2)$$

В коде $\varepsilon_{\text{turn}} = 0.05$ рад $\approx 2.9^\circ$ — поправка `mps.scenario.turn_tolerance_rad`.

Внимание

Wrap-around в $[-\pi, \pi]$ — важно! Если $\theta = +179^\circ$ и $\varphi = -179^\circ$, разность «по-наивному» = 358° — огромная, хотя физически они почти совпадают.

Поэтому используем функцию `_normalize_angle(a)`:

$$\text{wrap}_{[-\pi, \pi]}(a) = ((a + \pi) \bmod 2\pi) - \pi. \quad (11.3)$$

В коде:

```
1 def _normalize_angle(a: float) -> float:
2     return (a + math.pi) % (2 * math.pi) - math.pi
```

Без этого, в редких случаях, MPC получал бы reference-ошибку курса $\Delta\theta \sim 2\pi$ — умножал бы её на **K** — получал бы безумно большое u_ω — упирался в насыщение и крутился в обратную сторону.

11.2.4 Тайм-аут фазы TURN

Чтобы при поломанной IMU TURN не заиклился навсегда, есть собственный тайм-аут: $t_{\text{turn}} \leq 10$ с (параметр `mps.scenario.turn_timeout_s`). Не успели — завершаем прогон с `status='timeout'`.

11.3 Переход TURN → DRIVE

Когда условие завершения TURN сработало, `_tick_turn` возвращает True, и в `_tick` тем же тиком вызывается `_tick_drive(run, x)`. Между ними:

- не теряем тик (не пропускаем 20 мс на лоське);
- `run.phase = 'drive'` — фаза переключилась;
- `run.drive_t = 0.0` — часы DRIVE-фазы стартуют с нуля, отдельно от общего t (для `ramp s_ref` и DRIVE-тайм-аута).

mps_node.py:366, общий tick

```

1 def _tick(self):
2     with self._lock:
3         run = self._run
4         if run is None or self._fsm_state != 'DRIVE_FORWARD_MPS':
5             return
6         x = self._x_meas.copy()
7         # Относительные координаты.
8         x[_S] = x[_S] - run.s_start
9         x[_THETA] = _normalize_angle(x[_THETA] - run.theta_start)
10
11        # Двухфазный сценарий.
12        if run.phase == 'turn':
13            if not self._tick_turn(run, x):
14                return # ещё крутимся / прогон завершен
15            # TURN завершился ЭТИМ тиком → продолжаем в DRIVE
16        self._tick_drive(run, x)

```

11.4 Фаза DRIVE

После переключения в DRIVE логика та же, что в главе 10, с одной поправкой: курс reference — φ , а не 0.

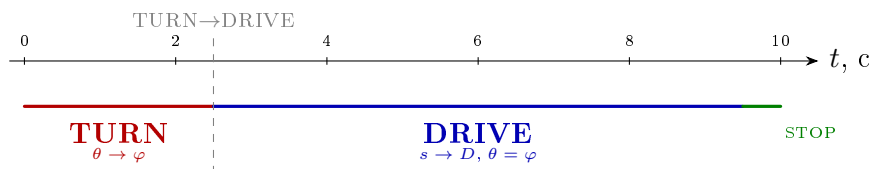
$$\mathbf{x}_{\text{drive}}^{\text{ref}}(k) = \begin{pmatrix} s_{\text{ref}}(k) \\ v_{\text{target}} \\ \varphi \\ 0 \\ 0 \end{pmatrix}, \quad s_{\text{ref}}(k) = \min(D, k \Delta t \cdot v_{\text{target}}).$$

Идея

Почему именно так, а не «следовать дуге»? Робот в DRIVE *уже* ориентирован к φ (фаза TURN об этом позаботилась). Он едет вперёд в своей системе координат, а курс держит постоянным. Это самая простая постановка, и она работает.

Если бы мы хотели ехать по *дуге* (например, для плавного объезда препятствия), то и reference, и линеаризация были бы сложнее: вместо постоянной опорной $(v_0, 0)^T$ надо было бы линеаризовать вдоль дуги с $\bar{\omega} \neq 0$. Это уже не учебный сценарий, а полноценная trajectory tracking.

11.5 Timeline прогона



Поля `_RunState`, обслуживающие двухфазность:

- `run.t` — монотонное *суммарное* время от старта прогона. Растёт всегда. Используется для тайм-аута фазы TURN и как абсцисса телеметрии.
- `run.drive_t` — часы только DRIVE-фазы, стартуют с 0 в момент переключения. Используются для `gamp sref` и DRIVE-тайм-аута.
- `run.phase` — 'turn' либо 'drive'.

11.6 Граничные случаи

Что если?

$\varphi = 0$. Условие завершения TURN срабатывает на первом же тике (если курс точно 0): $\theta_{\text{start}} = 0 \Rightarrow x[\theta] = 0 \Rightarrow |0 - 0| = 0 < \varepsilon_{\text{turn}}$. Сценарий *скипает* фазу TURN и идёт сразу в DRIVE. Это и есть «прямо вперёд» из предыдущей главы.

$\varphi = \pi$ (**поворот назад**). Худший случай: робот должен развернуться на 180° . Через wrap-around $\theta - \varphi$ всё ещё корректно (нормализуется в $[-\pi, \pi]$), но при $\theta = 0$, $\varphi = \pi$ ошибка ровно $\pm\pi$ — знак u_ω неопределён. На практике первый тик MPC выберет какое-то направление, робот начнёт крутиться, и дальше всё разрешится. Скорее всего проект зафиксирует $\varphi \in (-\pi, \pi]$, не *строгий* π , но валидация в `_on_scenario_run` допускает.

Watchdog в TURN. Если на каком-то тике нет одометрии > 3 тиков подряд, мы не можем достоверно судить, повернулся ли робот. TURN завершается error, FSM возвращается в IDLE.

11.7 Зачем это всё было нужно

Идея

Думаем учебно: какие были *альтернативы*?

(А) **Не разделять фазы, дать MPC сразу полную задачу с целевой точкой** ($D \cos \varphi, D \sin \varphi$). Тогда модель должна была бы быть *инвариантной* к ориентации — то есть линеаризация вокруг прямолинейного движения уже не работает; нужна нелинейная модель или linearization gain scheduling.

(Б) **Сначала повернуться, потом ехать — но через два разных регулятора.** TURN — простой П-регулятор на угле, DRIVE — MPC. Так у нас и было раньше (старый PID). Но это значит, что в коде *два* регулятора, и переключение между ними может давать всплеск по `cmd_vel`. Также — два разных набора тюнинга, два моделирования.

(В, **выбранный**) **Один и тот же MPC, разные reference и капы управления в каждой фазе.** Минусы: дополнительная сложность переключения. Плюсы: один контроллер — одна модель — единое поведение. Прозрачно для учебного эксперимента: те же **Q, R** работают в обеих фазах.

11.8 Что мы получили

- Сценарий стал **двухфазным**: $\text{TURN} \rightarrow \text{DRIVE}$.
- Используется тот же MPC, но с разными reference и капами: TURN режим — $u_v = 0$, $|u_\omega| \leq 1.0$; DRIVE режим — u_v свободно, $|u_\omega| \leq 0.5$.
- Переход — мгновенный, тем же тиком, без пропуска управления.
- Угол φ работает корректно через wrap-around в $[-\pi, \pi]$.
- Граничные случаи ($\varphi = 0$, watchdog) обрабатываются самой логикой.

Остался один вопрос: **откуда берётся φ** ? Это 3D-пикер цели — последняя интересная математика в этом учебнике.

Chapter 12

3D-пикер цели: математика

12.1 Вопрос

Пользователь хочет указать курс φ , на который робот развернётся перед движением. Как именно? Можно было бы поставить поле «введите угол в градусах», но это *неудобно* — человек редко интуитивно знает, как $+72^\circ$ соотносится с «вон туда, под небольшим углом влево».

Идея

3D-пикер — решение из `feat/mps-target-picker`. На экране показана 3D-сцена: робот в центре, окружность фиксированного радиуса вокруг него, маркер на окружности. Пользователь *кликает* в любую точку «пола» сцены — маркер переезжает на окружность в направлении клика, а угол φ автоматически вычисляется.

Это даёт мгновенную визуальную обратную связь и интуитивный ввод. Под капотом — немного линейной алгебры и одна функция `atan2`.

12.2 Геометрия и соглашения

В 3D-сцене Three.js (React Three Fiber) система координат другая, чем в мире робота. Аккуратно зафиксируем:

Мир робота		Three.js
x ось	вперёд от робота (curr. heading)	вправо от камеры
y ось	влево от робота	вверх (высота над полом)
z ось	вверх	к камере (от сцены к зрителю)

Робот «смотрит» вдоль своей $+x$ при `yaw = 0`. В Three угол сцены сделан так, что пол — плоскость $y = 0$ Three; камера слегка выше смотрит вниз.

Соответствие координат: мир (w_x, w_y) соответствует Three $(w_x, h, -w_y)$, где h — высота над полом (для маркера — константа `MARKER_HEIGHT`). Минус в третьей координате потому что Three- z направлен «к камере», а мировой y — «влево».

Примечание

Соглашение об углах: φ — относительный курс. $\varphi = 0$ — прямо вперёд (вдоль текущего уаw робота); $\varphi > 0$ — влево (CCW против часовой); $\varphi < 0$ — вправо. Это совпадает с соглашением ROS и нашим robot frame в коде.

12.3 Точка клика → угол

12.3.1 Формула

В Three.js «raycast» от позиции мыши даёт точку клика на полу (x_3, z_3) . Переводим в мировые $(w_x, w_y) = (x_3, -z_3)$. Угол — обычная atan2:

Угол от точки клика

$$\varphi = \text{atan2}(w_y, w_x) = \text{atan2}(-z_3, x_3). \quad (12.1)$$

Функция atan2 (есть в любой математической библиотеке) аккуратно ставит знаки и выдаёт результат в $(-\pi, \pi]$. Заодно не путается на $w_x = 0$ (где «наивный» $\arctan(w_y/w_x)$ дал бы деление на ноль).

compute_node/frontend/src/lib/targetAngle.ts

```

1 // Точка клика на полу сцены (Three-координаты x, z) → относительн
   ый курс  $\varphi$ .
2 export function groundPointToAngle(
3   threeX: number, threeZ: number
4 ): number {
5   // мир: wx = threeX, wy = -threeZ.  $\varphi = \text{atan2}(wy, wx)$ .
6   return Math.atan2(-threeZ, threeX)
7 }
```

12.3.2 Почему именно atan2?

Идея

Альтернатива — $\text{atan}(w_y/w_x)$. Беда:

- Делит на ноль при $w_x = 0$ (точно «влево» или «вправо»).
- Не различает квадранты: $\arctan(1/1) = \arctan(-1/-1) = \pi/4$, хотя углы — 45° и -135° .

atan2 принимает оба аргумента *отдельно*, использует знаки, покрывает все 4 квадранта и не падает на нуле. Это стандартный приём в графике и робототехнике.

12.3.3 Численные примеры

Пример

$(w_x, w_y) = (1, 0)$: точка прямо перед роботом. $\varphi = \text{atan2}(0, 1) = 0$ — «прямо».
 $(w_x, w_y) = (0, 1)$: точка слева от робота. $\varphi = \text{atan2}(1, 0) = \pi/2 \approx 90^\circ$.
 $(w_x, w_y) = (-1, 0)$: точка прямо за роботом. $\varphi = \text{atan2}(0, -1) = \pi$ — разворот на 180° .
 $(w_x, w_y) = (-1, -1)$: точка справа-сзади. $\varphi = \text{atan2}(-1, -1) = -3\pi/4 \approx -135^\circ$.

12.4 Угол $\rightarrow$ позиция маркера

Обратное: имея φ и фиксированный радиус N (радиус окружности в сцене), найти позицию маркера в Three-координатах.

Маркер на окружности радиуса N

$$(x_3, h, z_3) = (N \cos \varphi, h_{\text{marker}}, -N \sin \varphi). \quad (12.2)$$

Объяснение: в мировой системе точка на окружности радиуса N под углом φ — это $(N \cos \varphi, N \sin \varphi)$. Переходим в Three: $z_3 = -w_y = -N \sin \varphi$, $x_3 = w_x = N \cos \varphi$. Высота h_{marker} — константа над полом (чтобы маркер не тонул в пол).

lib/targetAngle.ts

```

1 export const MARKER_HEIGHT = 0.03
2
3 export function angleToMarkerPosition(
4   angle: number,
5   radius: number,
6 ): [number, number, number] {
7   return [radius * Math.cos(angle), MARKER_HEIGHT, -radius * Math.
      sin(angle)]
8 }

```

12.4.1 Почему радиус *фиксированный*?

В предыдущей версии (одна фаза, без TURN) пикер позволял указать *любую точку* — и расстояние, и направление. Это перегружало интерфейс: пользователь думает «куда» И «насколько далеко» одновременно.

В двухфазной схеме *дистанция D задаётся отдельным полем*, а пикер отвечает только за *направление*. Поэтому маркер всегда ползёт по окружности фиксированного радиуса N — независимо от того, куда конкретно кликнул пользователь. Это *разделение ответственности*: одна задача — один контрол.

12.5 Центральная мёртвая зона

Идея

Проблема: если кликнуть *рядом с центром*, маленькое движение мыши даёт огромный скачок φ . У точки $(0.01, 0.01)$ угол $= \pi/4$, у точки $(0.01, -0.01)$ угол $= -\pi/4$ — разница в 90° при микроперемещении в 1 см.

Решение: в зоне радиуса $r_{\text{dead}} = 0.3 N$ от центра *игнорируем* клик.

lib/targetAngle.ts

```
1 export const CENTER_DEADZONE_FRACTION = 0.3
2
3 export function groundPointRadius(
4   threeX: number, threeZ: number
5 ): number {
6   return Math.hypot(threeX, threeZ)
7 }
```

В UI-логике: если $\text{groundPointRadius} < \text{CENTER_DEADZONE_FRACTION} * N$, клик *не* обновляет φ .

12.6 Подпись угла

Финальный штрих — человекочитаемая подпись маркера: «прямо», « $+35^\circ$ », « -40° »:

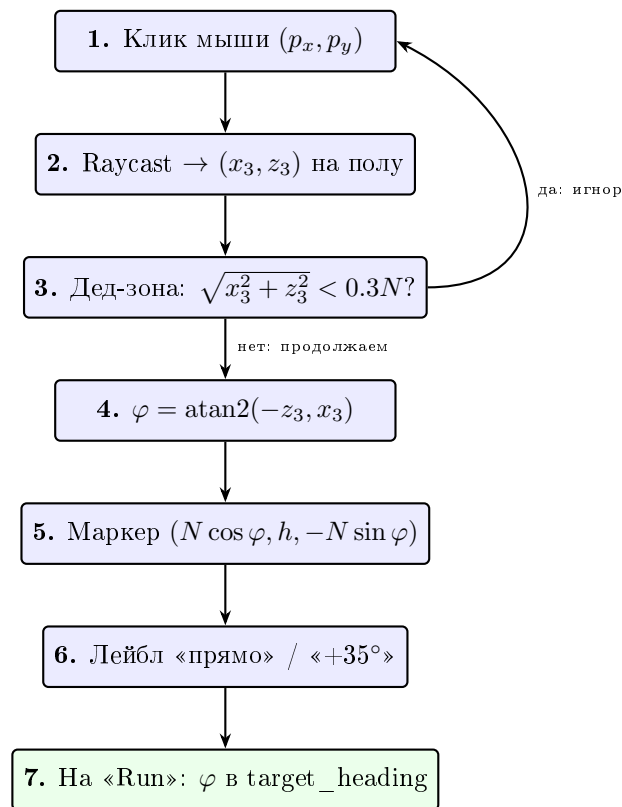
lib/targetAngle.ts

```
1 export function formatHeadingLabel(angle: number): string {
2   const deg = (angle * 180) / Math.PI
3   if (Math.abs(deg) < 3) return 'прямо'
4   const rounded = Math.round(deg)
5   return rounded > 0 ? `+${rounded}°` : `-${rounded}°`
6 }
```

Дед-зона по *углу* ($\leq 3^\circ$ показывается как «прямо») — не путать с дед-зоной по *радиусу*. Это разные защиты:

- Центральная (радиус) — против скачков при микроперемещениях.
- Угловая (3°) — косметика лейбла, чтобы не мигало $+1^\circ / -1^\circ$ при дрожании руки.

12.7 Полный пайплайн пикера



12.8 От UI к роботу

Когда пользователь жмёт «Run on Robot», значение φ уходит в REST-запрос:

REST

```

POST /api/v1/mps/scenario/run
Content-Type: application/json

{
  "source": "robot",
  "request": {
    "distance": 2.0,
    "v_target": 0.15,
    "target_heading": 0.7854  // <-- phi (угол) из пикера, рад
  }
}
```

`routers/mps.py` публикует это в MQTT `mps/scenario/run`, `mps_node` читает, валидирует ($-\pi \leq \varphi \leq \pi$), ставит `run.target_heading = \varphi`, фаза 'turn' — и поехали.

12.9 Тесты математики пикера

Файл `compute_node/frontend/src/lib/targetAngle.test.ts` (см. в проекте) покрывает:

- Прямые тестовые случаи: $(1, 0) \rightarrow 0$, $(0, -1) \rightarrow \pi/2$ (помним Three's z перевёрнут), $(-1, 0) \rightarrow \pi$ и т.д.
- Симметрию: `angleToMarkerPosition(groundPointToAngle(x, z) | radius = N)` возвращает точку на окружности радиуса N в том же направлении, что исходный клик.
- Граничные значения подписи: $\pm 2.9^\circ \rightarrow$ «прямо», $\pm 3.1^\circ \rightarrow$ числовая подпись.
- Дед-зону по радиусу.

Юнит-тесты в jsdom без WebGL — именно потому модуль `targetAngle.ts` не зависит от Three: вся «математика» вынесена в чистые функции, а 3D-визуал — отдельным компонентом `MpsTargetScene.tsx`.

12.10 Что мы получили

- Пикер цели — интуитивный визуальный ввод угла φ .
- Математика тривиальная: $\varphi = \text{atan2}(-z_3, x_3)$ для клика $\rightarrow$ угла, $(N \cos \varphi, h, -N \sin \varphi)$ для угла $\rightarrow$ позиции.
- Соглашения о координатах фиксированы и совпадают с тем, что потом получает Pi.
- Дед-зона по радиусу защищает от скачков, дед-зона по углу — косметика.
- Юнит-тестируется без WebGL — спасибо чистым функциям.

Это была вся «активная» математика модуля МПС. Дальше осталась ещё одна, инфраструктурная глава — как матрицы из MATLAB попадают в Python.

Chapter 13

Реализация: путь чисел

13.1 Вопрос

Все предыдущие главы говорили: «матрица $\mathbf{A}$ имеет вот такой вид», « $\mathbf{Q}$ выбирают так-то». Но как конкретное число -6.667 из (6.5) попадает в Python-код, который шлёт команды мотору? Где это происходит, и что меняется на каждом шаге?

Идея

В проекте — классическая трёхэтапная схема:

1. **Offline в MATLAB:** синтез матриц (модель, веса, проверка устойчивости).
2. **Экспорт в YAML:** матрицы пишутся в `config.yaml` как обычные списки чисел.
3. **Online в Python:** `mps_node` читает YAML при старте, строит `StateSpaceModel` и `MPCController`, гонит тики.

Плюс **live-канал** от UI: пользователь меняет матрицы в браузере $\rightarrow$ MQTT `mps/matrices/set` $\rightarrow$ `mps.rebuild()` на лету.

13.2 Этап 1: MATLAB (`matlab/main.m`)

В `matlab/` лежат оффлайн-скрипты для синтеза матриц. Логика стандартная:

1. Загрузить параметры объекта $(v_0, \tau_v, \tau_\omega)$ из конфига.
2. Построить $\mathbf{A}, \mathbf{B}$ по формулам (6.3), (6.4).
3. Проверить управляемость: `rank(ctrb(A, B)) == n`.
4. Выбрать $\mathbf{Q}, \mathbf{R}$.
5. Решить DARE (MATLAB-функция `dlqr/dare`) — получить $\mathbf{P}, \mathbf{K}$.
6. Визуализировать `step response`: убедиться, что замкнутая система устойчива и качественно ведёт себя.
7. Экспортировать всё в YAML.

matlab/main.m (упрощённый эскиз)

```

1  % Параметры объекта
2  v0    = 0.20;
3  tau_v = 0.15;
4  tau_w = 0.10;
5  Ts    = 0.02;      % период МПС-цикла, 50 Гц
6
7  % Непрерывные матрицы
8  A = [0 0 0 1 0;
9       0 0 v0 0 0;
10      0 0 0 0 1;
11      0 0 0 -1/tau_v 0;
12      0 0 0 0 -1/tau_w];
13  B = [0 0; 0 0; 0 0; 1/tau_v 0; 0 1/tau_w];
14
15  % Управляемость
16  assert(rank(ctrb(A, B)) == 5);
17
18  % Дискретизация
19  sysd = c2d(ss(A, B, eye(5), zeros(5,2)), Ts, 'zoh');
20  Ad = sysd.A;  Bd = sysd.B;
21
22  % Веса
23  Q = diag([10 10 5 1 1]);
24  R = diag([1 1]);
25
26  % LQR / DARE
27  [K, P, ~] = dlqr(Ad, Bd, Q, R);
28
29  % Проверка устойчивости замкнутой
30  assert(all(abs(eig(Ad - Bd*K)) < 1));
31
32  % Экспорт
33  export_to_yaml(A, B, Q, R, K, P, 'config.yaml');

```

Примечание

MATLAB используется потому что: (1) исторически он стандарт ТАУ-курсов, (2) c2d, dlqr, step, графики — встроенные и предсказуемые, (3) учебник Козлова пишет в MATLAB-нотации.

Альтернатива — Python+scipy.signal+matplotlib. Сейчас проект использует MATLAB для синтеза и Python — для онлайн-управления. В tools/ есть Python-эквивалент для тех, у кого нет лицензии MATLAB.

13.3 Этап 2: YAML — источник правды

config.yaml — общая «истина» для Python и UI. В нём *две независимые* секции для двух моделей:

config.yaml, секция control (легаси)

```
control:
  plant:
    Ts: 0.05
    v0: 0.20
    tau_v: 0.15
    tau_w: 0.10
  matrices:
    A: [[0, 0, 0, 1, 0],
        [0, 0, 0.2, 0, 0],
        [0, 0, 0, 0, 1],
        [0, 0, 0, -6.667, 0],
        [0, 0, 0, 0, -10]]
    B: [[0, 0],
        [0, 0],
        [0, 0],
        [6.667, 0],
        [0, 10]]
    K_mpc: [...] # предвычисленное усиление
    Pf: [...] # терминальный штраф
  weights:
    Q_diag: [10.0, 10.0, 5.0, 1.0, 1.0]
    R_diag: [1.0, 1.0]
  mpc:
    horizon_N: 10
    solver: "clip"
  limits:
    u_min: [-0.30, -2.0]
    u_max: [+0.30, +2.0]
```

config.yaml, секция mps (учебная)

```
mps:
  plant:
    Ts: 0.02 # 50 Гц
  matrices:
    A: [...] # непрерывная (контракт api.md)
    B: [...] # непрерывная
    C: [...] # для UI визуализации (по умолч. I)
    D: [...] # для UI визуализации (по умолч. 0)
  weights:
    Q_diag: [...]
    R_diag: [...]
  horizon_N: 20
  limits:
    u_min: [...]
    u_max: [...]
```

```

scenario:
  distance_max:      5.0
  v_target_max:      0.30
  omega_max_in_forward: 0.5
  omega_max_in_turn:  1.0
  turn_tolerance_rad: 0.05
  turn_timeout_s:     10.0
  default_distance:   2.0
  default_v_target:   0.15

```

Внимание

Две секции не пересекаются. `control.*` — для старого PID-/легаси-MPC контура; `mps.*` — для учебного МПС-модуля. *Не путать:* легаси-секция содержит *дискретные* матрицы и готовое усиление `K_mps`, а МПС-секция — *непрерывные* матрицы, и ЗОН-дискретизация делается в Python при загрузке. Если случайно подать `mps`-матрицы в легаси-контур (или наоборот), размерности совпадут (по 5×5), но смысл компонент состояния — разный, и MPC «закрутит» работа не туда. Так и было в коммите 8521742 до его починки.

13.4 Этап 3: Python (StateSpaceModel, MPCController)

При старте `mps_node` читает `config.yaml` и собирает `plant` + MPC:

`pi_nodes/nodes/mps_node.py:180`, метод `_build_from_config`

```

1  def _build_from_config(self):
2      A = self._cfg('mps.matrices.A', None)
3      B = self._cfg('mps.matrices.B', None)
4      C = self._cfg('mps.matrices.C', None)
5      D = self._cfg('mps.matrices.D', None)
6      Q_diag = self._cfg('mps.weights.Q_diag', None)
7      R_diag = self._cfg('mps.weights.R_diag', None)
8      N = self._cfg('mps.horizon_N', None)
9      u_min = self._cfg('mps.limits.u_min', None)
10     u_max = self._cfg('mps.limits.u_max', None)
11     if any(v is None for v in (A, B, Q_diag, R_diag, N, u_min,
12                               u_max)):
13         raise RuntimeError('config.yaml: секция mps: неполна')
14
15     A = np.asarray(A, dtype=float)
16     B = np.asarray(B, dtype=float)
17     Ad, Bd = zoh_discretize(A, B, self._mps_ts)
18     plant = StateSpaceModel(Ad=Ad, Bd=Bd, Cd=C, Dd=D, Ts=self._mps_ts)
19     mpc = MPCController(
20         Ad=Ad, Bd=Bd,
21         Q=np.diag(np.asarray(Q_diag, dtype=float)),
22         R=np.diag(np.asarray(R_diag, dtype=float)),
23         N=int(N),

```

```

23     u_min=np.asarray(u_min, dtype=float),
24     u_max=np.asarray(u_max, dtype=float),
25     solver='clip',
26 )
27     return plant, mpc

```

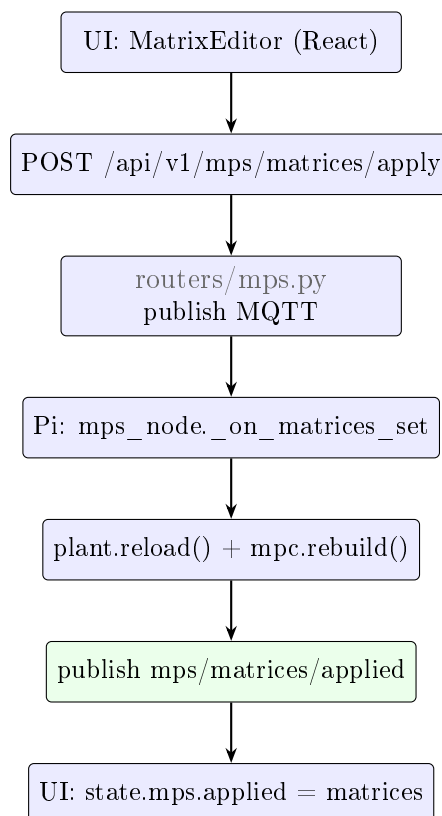
Что важно отметить:

- Матрицы из YAML — *непрерывные*; ЗОН-дискретизация происходит *здесь*, в одной точке кода.
- A_d , B_d затем используются u в plant (StateSpaceModel), u в MPC — единый источник правды. Симулятор на ноуте делает то же самое (compute_node/mps_runner.py: _build_controller) — три места, одна логика.
- solver='clip' зашит — учебный, быстрый, не требует scipy на Pi.

13.5 Live Apply: четвёртый канал

Сверх старта-из-конфига есть **live-канал**: пользователь редактирует A, B, Q, R в UI и жмёт Apply.

13.5.1 Поток данных



Ключевые моменты:

- Запрос отбрасывается, если прогон идёт (`is_running == True`). Apply — *между* прогонами.
- На Pi A , B приходят в непрерывной форме — ZOH-дискретизируем тем же `zoh_discretize`, что и на старте.
- `rebuild()` — атомарный: `snapshot` всех полей, при ошибке валидации (например, R не положительная) — `rollback` к снимку. См. главу 9, §`rebuild`.
- `Ack mps/matrices/applied` приходит обратно в UI — пользователь видит «зелёную галочку».

13.5.2 Что валидирует Pi

1. Формы матриц: A квадратная, B совместима с A , Q диагональная нужного размера, и т.д.
2. Положительность: $r_i > 0$. Иначе DARE не сойдётся.
3. Размер $N \geq 1$, $u_{\min} < u_{\max}$.
4. Замкнутая система устойчива (опционально, в `mps.is_stable()`).

При нарушении публикуется `mps/error`, тип `error_type="precondition"`. UI показывает алерт.

13.6 Согласование sim и работа

Идея

Кошмарный сценарий: симулятор и робот строят MPC *по-разному*. В `sim` матрицы дискретизируются с $T_s = 0.02$, на роботе — 0.05. Или симулятор берёт Q из YAML, а робот — свои дефолты. Тогда *прогон в sim* перестаёт быть предсказанием реального прогона — учебный модуль теряет смысл.

Решение: *три* места, где строится контроллер (на старте Pi, при Apply на Pi, в симуляторе), должны быть **строго идентичны** по логике. Это инвариант, который тесты охраняют (`tests/test_mps_runner.py`, `tests/test_mps_node.py`).

13.7 Тесты

Файл	Что проверяет	Где живёт
<code>test_mps_runner.py</code>	Sim: propagation, reach, метрики	<code>tests/</code>
<code>test_mps_router.py</code>	REST/WS контракты, history	<code>tests/</code>
<code>test_mps_node.py</code>	MQTT handlers, FSM, abort	<code>tests/</code>
<code>test_state_space_extended.py</code>	Cd, Dd, <code>reload()</code> , <code>output()</code>	<code>tests/</code>
<code>test_mpc_controller.py</code>	<code>rebuild()</code> , горизонт, веса, <code>rollback</code>	<code>tests/</code>
<code>targetAngle.test.ts</code>	Математика пикера (<code>jsdom</code>)	<code>compute_node/frontend/</code>
<code>MpsPage.test.tsx</code>	UI flow: пикер, Run, история	<code>compute_node/frontend/</code>

Coverage таргет — 85% по проекту. CI на каждый PR в feat/mps прогоняет все эти суиты, плюс *smoke-тест на реальном Pi (A10)* — руками, по чек-листу.

13.8 Где какие риски

Примечание

Учебная природа модуля = вольный экспериментатор у руля. Места, где *реально* может «не пойти», и куда смотреть в первую очередь:

- **R с нулём.** DARE не сойдётся, rebuild() упадёт. Pi ответит precondition.
- **Q с большими разбросами весов.** Может дать численно плохо обусловленную **H**, **K** будет огромным → робот будет дёргаться. Pre-validate этого не ловит, ловит is_stable().
- **Несовпадение секций конфига и трактовки.** См. warning выше про коммит 8521742.
- **Watchdog по одометрии.** Если выдернуть USB у Pi от мотор-контроллера во время прогона — watchdog отработает за $3 \cdot T_s = 60$ мс.

13.9 Что мы получили

- Числа модели проходят через три точки: MATLAB (синтез) — YAML (источник правды) — Python (онлайн).
- Кроме старта-из-конфига, есть live-канал: UI -> MQTT -> mpc.rebuild(). Между прогонами, атомарно, с rollback.
- Три места построения контроллера (Pi-start, Pi-Apply, simulator) обязаны быть идентичны — это инвариант.
- Тестов — семь файлов, покрывающих все три уровня (юнит на backend, юнит на frontend, smoke на Pi).

Поздравляем. От «нажал кнопку» до ШИМ-сигнала в моторе — вся математика, вся архитектура, все подводные камни. Если ты читал внимательно, ты теперь можешь:

1. Объяснить другому, что значит «вектор состояния».
2. Вывести **A**, **B** из физической постановки.
3. Понимать, что делает каждая строка mpc_node._tick().
4. Предсказать, как изменится поведение робота при изменении **Q**, **R**, **N**.
5. Найти и починить баг, аналогичный коммиту 8521742.

Удачи с курсовой.